

Asset Management - Group Project

Luca Silva, Gabriel Lopez, Cyril Zaki

2025-12-17

Contents

1	Workshop 1 — Robust Global Minimum Variance Portfolio	3
1.1	Introduction	3
1.2	Data and wrangling	4
1.3	Exploratory data analysis	6
1.4	Methodology and implementation	7
1.5	Covariance estimators and GMV portfolios	9
1.6	Results and interpretation	11
1.7	Conclusion	17
2	Workshop 2 — Review with Dynamic Allocation and Machine Learning views	18
2.1	Introduction	18
2.2	Data and Preprocessing	19
2.3	Rebalancing framework and core implementation helpers	21
2.4	Benchmark strategy: equal-weight (buy-and-hold with monthly rebalancing)	23
2.5	Markowitz benchmark with shrinkage, rolling estimation, and stabilized weights	24
2.6	Results (Part I): equal-weight benchmark vs tuned Markowitz strategy	28
2.7	Backtest diagnostics and robustness analysis	30
2.8	Appendix (Python): XGBoost pipeline and anti-bias design	35
2.9	Machine learning signals and integration into Black–Litterman	37
3	Workshop 3 — Portfolio Insurance Strategies : OBPI vs CPPI	49
3.1	Strategies Overview	49
3.2	Modeling	51
3.3	Simulation of OBPI and CPPI	53
3.4	Monte Carlo Simulation	56
3.5	General Conclusion	62

1 Workshop 1 — Robust Global Minimum Variance Portfolio

1.1 Introduction

This report replicates and extends the baseline framework introduced in Workshop 1 – Equity portfolio choice, Global Minimum Variance portfolio and factor models (Clauss, 2025). The workshop constructs the Global Minimum Variance (GMV) portfolio in the Markowitz framework and illustrates how covariance estimation affects the stability of optimal weights.

The main contribution of this extension is to evaluate the robustness of the GMV portfolio under multiple covariance estimators and to communicate results as in an academic empirical project: (i) clear methodology and formulas, (ii) out-of-sample evaluation (ex-ante vs ex-post), and (iii) diagnostic analysis of weight concentration and stability.

Let Σ denote the covariance matrix of asset returns and e a vector of ones of dimension n . The GMV portfolio is defined by

$$\omega_{\text{GMV}} = \frac{\Sigma^{-1}e}{e^\top \Sigma^{-1}e}.$$

file:///C:/Users/lucas/Desktop/Workshop1.pdf and its ex-ante (theoretical) volatility is

$$\sigma_{\text{GMV}} = \sqrt{\frac{1}{e^\top \Sigma^{-1}e}}.$$

.

The central empirical issue is that Σ is unknown and must be estimated from a finite sample. Because the GMV weights involve Σ^{-1} , estimation noise can be amplified, leading to unstable allocations and poor out-of-sample volatility. We therefore compare the baseline sample covariance estimator to two regularization approaches: PCA factor filtering (Sharpe-style factor structure) and covariance shrinkage (Ledoit and Wolf, 2004).

1.2 Data and wrangling

We download daily prices from Yahoo Finance (via tidyquant) and compute arithmetic monthly returns. Monthly sampling is coherent with the workshop logic: returns are closer to Gaussian than daily returns, which aligns better with the classical Markowitz setting.

```
from_date <- "2006-05-01"
to_date   <- "2025-08-31"

symbols <- c(
  "AI.PA", "CS.PA", "BNP.PA", "EN.PA", "CA.PA", "BN.PA", "ENGI.PA",
  "EL.PA", "LR.PA", "OR.PA", "MC.PA", "ML.PA", "RI.PA", "SGO.PA",
  "SAN.PA", "SU.PA", "GLE.PA", "STLAP.PA", "TTE.PA", "VIE.PA",
  "CAP.PA", "AIR.PA", "KER.PA", "ORA.PA", "UG.PA", "VIV.PA", "ACA.PA"
)

stock_prices <- tq_get(
  symbols,
  from = from_date,
  to   = to_date,
  get  = "stock.prices"
)

monthly_returns <- stock_prices %>%
  group_by(symbol) %>%
  tq_transmute(
    select = adjusted,
    mutate_fun = periodReturn,
    period = "monthly",
    type = "arithmetic",
    col_rename = "monthly.returns"
  )

table_returns <- monthly_returns %>%
  pivot_wider(names_from = symbol, values_from = monthly.returns) %>%
  select(-date) %>%
  na.omit()

T_all <- nrow(table_returns)
n_all <- ncol(table_returns)

cat("Sample size (months) T =", T_all, "\n")

## Sample size (months) T = 232
```

```
cat("Universe size N =", n_all, "\n")
```

```
## Universe size N = 26
```

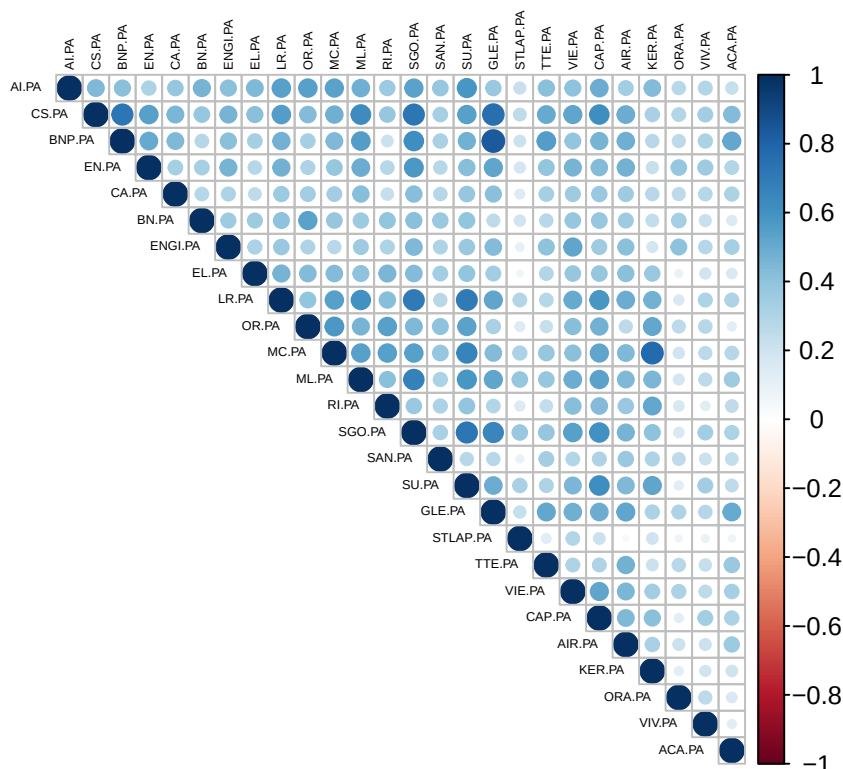
1.3 Exploratory data analysis

We summarize a few series and visualize the correlation structure. The key point for GMV is that correlations are clearly non-zero, so diversification matters, but they are not perfect, so an interior optimum exists.

```
summary(table_returns[, 1:min(6, n_all)])
```

```
##      AI.PA      CS.PA      BNP.PA      EN.PA
## Min.   :-0.13454  Min.   :-0.39935  Min.   :-0.371703  Min.   :-0.246900
## 1st Qu.: -0.01769  1st Qu.: -0.03356  1st Qu.: -0.049140  1st Qu.: -0.035168
## Median :  0.01178  Median :  0.01488  Median :  0.013508  Median :  0.006217
## Mean   :  0.01002  Mean   :  0.01144  Mean   :  0.008645  Mean   :  0.006182
## 3rd Qu.:  0.03687  3rd Qu.:  0.05880  3rd Qu.:  0.067126  3rd Qu.:  0.051570
## Max.   :  0.13924  Max.   :  0.42841  Max.   :  0.439122  Max.   :  0.268997
##      CA.PA      BN.PA
## Min.   :-0.2430708  Min.   :-0.207393
## 1st Qu.: -0.0453627  1st Qu.: -0.022308
## Median :  0.0031781  Median :  0.006851
## Mean   : -0.0006946  Mean   :  0.005468
## 3rd Qu.:  0.0430806  3rd Qu.:  0.037865
## Max.   :  0.1592890  Max.   :  0.152109
```

```
corrplot(cor(table_returns), type = "upper", tl.col = "black", tl.cex = 0.45)
```



1.4 Methodology and implementation

A key difficulty in the Global Minimum Variance framework is that the covariance matrix Σ is unknown and must be estimated from a finite sample. Since GMV portfolio weights depend on the inverse covariance matrix Σ^{-1} , even small estimation errors in $\hat{\Sigma}$ may be strongly amplified, leading to unstable allocations and disappointing out-of-sample performance. This issue is commonly referred to as estimation risk and constitutes the main practical limitation of mean–variance portfolio optimization. The objective of this project is therefore not to modify the Markowitz framework itself, but to evaluate how different covariance estimators mitigate estimation risk and affect the robustness of GMV portfolios in empirical applications.

The objective of this project is therefore not to “improve” Markowitz theory, but to assess how different covariance estimators mitigate estimation risk and affect the stability and robustness of GMV portfolios in practice. We split the sample into a learning period (estimation) and a backtest period (evaluation). Volatilities are annualized with factor $\sqrt{12}$.

We compare the following approaches:

- Empirical GMV (baseline workshop estimator with unbiased correction when feasible).
- PCA-GMV with 1 factor (low-rank systematic structure + diagonal idiosyncratic residual).
- PCA-GMV with 3 factors.
- Shrinkage-GMV (Ledoit–Wolf style shrinkage via `cov.shrink`).

Two benchmarks are included: equal-weighted $1/N$ and a simple long-only projection of empirical GMV weights.

```
perio <- 12
cutoff <- round(T_all * 0.7)

learning <- table_returns[1:cutoff, , drop = FALSE]
backtest <- table_returns[(cutoff + 1):T_all, , drop = FALSE]

T_learn <- nrow(learning)
n <- ncol(learning)
e <- rep(1, n)

solve_sym_pd <- function(Sigma, b = NULL) {
  if (is.null(b)) solve(Sigma) else drop(solve(Sigma, b))
}

gmw_from_cov <- function(Sigma, e) {
  z <- solve_sym_pd(Sigma, e)
  C <- sum(e * z)
  list(weights = z / C, sigma_ante = sqrt(1 / C))
}

cov_pca_k <- function(Sigma, k) {
  eg <- eigen(Sigma, symmetric = TRUE)
```

```

vec <- eg$vectors[, 1:k, drop = FALSE]
lam <- eg$values[1:k]
var_res <- diag(Sigma) - as.vector((vec^2) %*% lam)
var_res <- pmax(var_res, 1e-12)
vec %*% diag(lam, k, k) %*% t(vec) + diag(var_res, nrow(Sigma))
}

expost_sigma <- function(R, w, perio = 12) {
sd(as.matrix(R) %*% w) * sqrt(perio)
}

# Concentration diagnostics

herfindahl <- function(w) sum(w^2)
top_k_table <- function(w, assets, k = 10) {
tibble(Asset = assets, Weight = as.numeric(w)) %>%
arrange(desc(Weight)) %>%
slice(1:k)
}

```

1.5 Covariance estimators and GMV portfolios

1.5.1 Empirical covariance (baseline)

```
if (T_learn > n + 2) {  
  Sigma_emp <- cov(learning) * (T_learn - 1) / (T_learn - n - 2) * perio  
} else {  
  Sigma_emp <- cov(learning) * perio  
}  
  
res_emp <- gmv_from_cov(Sigma_emp, e)  
omega_emp <- as.numeric(res_emp$weights)  
sig_ante_emp <- res_emp$sigma_ante  
sig_post_emp <- expost_sigma(backtest, omega_emp, perio)
```

1.5.2 PCA covariance (1 and 3 factors)

The PCA covariance estimator uses a low-rank systematic component plus a diagonal residual covariance:

$$\widehat{\Sigma}^{(k)} = \Phi_k \Lambda_k \Phi_k^\top + \Sigma_\varepsilon,$$

where Φ_k contains the first k eigenvectors and Λ_k the first k eigenvalues. The diagonal matrix Σ_ε keeps asset-specific residual variances.

```
Sigma_pca1 <- cov_pca_k(Sigma_emp, 1)  
res_pca1 <- gmv_from_cov(Sigma_pca1, e)  
omega_pca1 <- as.numeric(res_pca1$weights)  
sig_ante_pca1 <- res_pca1$sigma_ante  
sig_post_pca1 <- expost_sigma(backtest, omega_pca1, perio)  
  
Sigma_pca3 <- cov_pca_k(Sigma_emp, min(3, n))  
res_pca3 <- gmv_from_cov(Sigma_pca3, e)  
omega_pca3 <- as.numeric(res_pca3$weights)  
sig_ante_pca3 <- res_pca3$sigma_ante  
sig_post_pca3 <- expost_sigma(backtest, omega_pca3, perio)
```

1.5.3 Shrinkage covariance (Ledoit–Wolf)

Shrinkage regularizes $\widehat{\Sigma}$ toward a structured target to reduce estimation error in high dimension. This follows the idea formalized in Ledoit and Wolf (2004): accept a small bias to reduce variance of the estimator and stabilize $\widehat{\Sigma}^{-1}$.

Covariance shrinkage provides an explicit solution to the bias–variance trade-off inherent in covariance estimation. By shrinking the sample covariance matrix toward a structured target, the

estimator deliberately introduces a small bias in exchange for a substantial reduction in estimation variance. As shown by Ledoit and Wolf (2004), this regularization stabilizes the inverse covariance matrix and, consequently, the GMV portfolio weights. The empirical relevance of shrinkage therefore depends on whether the reduction in estimation variance dominates the induced bias, which justifies a systematic out-of-sample evaluation.

Formally, the GMV weights associated with the shrinkage estimator are given by

$$\omega_{\text{GMV}}^{\text{shr}} = \frac{\widehat{\Sigma}_{\text{shr}}^{-1} e}{e^\top \widehat{\Sigma}_{\text{shr}}^{-1} e},$$

and the corresponding ex-ante volatility is

$$\sigma_{\text{GMV}}^{\text{shr}} = \sqrt{\frac{1}{e^\top \widehat{\Sigma}_{\text{shr}}^{-1} e}}.$$

```
Sigma_shr <- cov.shrink(learning) * perio

## Estimating optimal shrinkage intensity lambda.var (variance vector): 0.4785
##
## Estimating optimal shrinkage intensity lambda (correlation matrix): 0.0603

res_shr <- gmv_from_cov(Sigma_shr, e)
omega_shr <- as.numeric(res_shr$weights)
sig_ante_shr <- res_shr$sigma_ante
sig_post_shr <- expost_sigma(backtest, omega_shr, perio)
```

1.5.4 Benchmarks and constraints

The equal-weighted $1/N$ portfolio is included as a benchmark following DeMiguel, Garlappi, and Uppal (2009), who document that naive diversification often performs competitively relative to optimized portfolios in the presence of estimation error. In addition, a simple long-only projection of the empirical GMV portfolio is considered to illustrate the stabilizing role of constraints, in line with the intuition developed by Jagannathan and Ma (2003). These benchmarks provide economically meaningful reference points for evaluating the benefits of covariance regularization.

The equal-weighted portfolio is defined as

$$\omega_i^{1/N} = \frac{1}{N}, \quad i = 1, \dots, N.$$

Because it does not rely on any parameter estimation, the $1/N$ portfolio is fully immune to estimation risk and provides a natural robustness benchmark for GMV-based allocations.

```
omega_1N <- rep(1/n, n)
sig_post_1N <- expost_sigma(backtest, omega_1N, perio)

omega_emp_longonly <- pmax(omega_emp, 0)
omega_emp_longonly <- omega_emp_longonly / sum(omega_emp_longonly)
sig_post_longonly <- expost_sigma(backtest, omega_emp_longonly, perio)
```

1.6 Results and interpretation

1.6.1 Ex-ante vs Ex-post volatility

The evaluation strategy adopted in this study relies on a systematic comparison between ex-ante and ex-post portfolio volatility. The ex-ante volatility σ_{GMV} represents the theoretical minimum variance implied by the estimated covariance matrix, while the ex-post volatility is computed from realized portfolio returns on an out-of-sample period. A large discrepancy between these two quantities is symptomatic of overfitting and excessive sensitivity to estimation noise. Consequently, a covariance estimator is considered robust not only if it yields a low in-sample volatility, but if it produces a small and stable ex-ante / ex-post volatility gap.

```
Results <- tibble(
  Model = c("Empirical", "PCA 1", "PCA 3", "Shrinkage", "Empirical Long-only", "1/N"),
  Sigma_ante = c(sig_ante_emp, sig_ante_pca1, sig_ante_pca3, sig_ante_shr, NA_real_, NA_real_),
  Sigma_post = c(sig_post_emp, sig_post_pca1, sig_post_pca3, sig_post_shr, sig_post_longonly, sig_post_1N),
  HHI = c(herfindahl(omega_emp), herfindahl(omega_pca1), herfindahl(omega_pca3), herfindahl(omega_shr),
  herfindahl(omega_emp_longonly), herfindahl(omega_1N)),
  MaxWeight = c(max(omega_emp), max(omega_pca1), max(omega_pca3), max(omega_shr),
  max(omega_emp_longonly), max(omega_1N))
)

knitr::kable(Results, digits = 4, caption = "Comparison of GMV estimators (volatility and concentration diagnostics)")
```

Table 1: Comparison of GMV estimators (volatility and concentration diagnostics)

Model	Sigma_ante	Sigma_post	HHI	MaxWeight
Empirical	0.1143	0.1403	0.2378	0.2176
PCA 1	0.0855	0.1234	0.1843	0.1760
PCA 3	0.0969	0.1337	0.2463	0.2038
Shrinkage	0.1220	0.1524	0.2058	0.2014
Empirical Long-only	NA	0.1478	0.0915	0.1469
1/N	NA	0.1788	0.0385	0.0385

PCA-based covariance estimators impose a low-rank structure on $\hat{\Sigma}$, consistent with the idea that asset returns are driven by a limited number of common risk factors. This approach is closely related to the single-index model introduced by Sharpe (1963) and later generalized in multi-factor asset pricing models. The one-factor specification primarily captures the dominant market-wide component of risk, while the three-factor specification allows for additional systematic sources of co-movement, such as sectoral or style effects. By filtering out high-dimensional noise, PCA reduces the effective dimensionality of the covariance matrix and stabilizes its inversion.

The empirical GMV portfolio provides a natural baseline consistent with the workshop. However, because $\hat{\Sigma}$ is estimated from a finite sample and then inverted, the portfolio can become sensitive to estimation noise, which is typically visible as a gap between ex-ante volatility σ_{GMV} and the realised out-of-sample volatility. PCA-based estimators reduce dimensionality by filtering the covariance

matrix through a small number of systematic components; this tends to stabilize weights when the dominant risk structure is indeed factor-driven. Shrinkage explicitly regularizes the covariance estimate and can further reduce instability, although its out-of-sample benefit is empirical and depends on the data and the chosen target.

To complement volatility comparisons, we report concentration diagnostics: the Herfindahl–Hirschman index $\sum_i \omega_i^2$ (higher means more concentrated) and the maximum single-asset weight. These diagnostics help interpret whether “improved volatility” comes at the cost of extreme allocations.

1.6.2 Weight tables (per asset) and barplots

We report the top 10 weights for each method, as in the workshop, and provide a visual comparison.

```
assets <- colnames(learning)

Top10_emp <- top_k_table(omega_emp, assets, k = 10)
Top10_pca1 <- top_k_table(omega_pca1, assets, k = 10)
Top10_pca3 <- top_k_table(omega_pca3, assets, k = 10)
Top10_shr <- top_k_table(omega_shr, assets, k = 10)

knitr::kable(Top10_emp, digits = 4, caption = "Top 10 weights - Empirical GMV")
```

Table 2: Top 10 weights — Empirical GMV

Asset	Weight
EL.PA	0.2176
TTE.PA	0.1844
LR.PA	0.1824
ORA.PA	0.1586
BN.PA	0.1537
AI.PA	0.0853
VIV.PA	0.0826
OR.PA	0.0771
SAN.PA	0.0664
CA.PA	0.0622

```
knitr::kable(Top10_pca1, digits = 4, caption = "Top 10 weights - PCA 1-factor GMV")
```

Table 3: Top 10 weights — PCA 1-factor GMV

Asset	Weight
AI.PA	0.1760
BN.PA	0.1651

Asset	Weight
OR.PA	0.1491
EL.PA	0.1456
TTE.PA	0.1280
SAN.PA	0.1123
ORA.PA	0.1112
RI.PA	0.0723
ENGL.PA	0.0633
VIV.PA	0.0613

```
knitr::kable(Top10_pca3, digits = 4, caption = "Top 10 weights - PCA 3-factor GMV")
```

Table 4: Top 10 weights — PCA 3-factor GMV

Asset	Weight
AL.PA	0.2038
BN.PA	0.1889
OR.PA	0.1661
EL.PA	0.1564
TTE.PA	0.1497
ORA.PA	0.1269
SAN.PA	0.1209
STLAP.PA	0.0995
VIV.PA	0.0628
ENGL.PA	0.0608

```
knitr::kable(Top10_shr, digits = 4, caption = "Top 10 weights - Shrinkage GMV")
```

Table 5: Top 10 weights — Shrinkage GMV

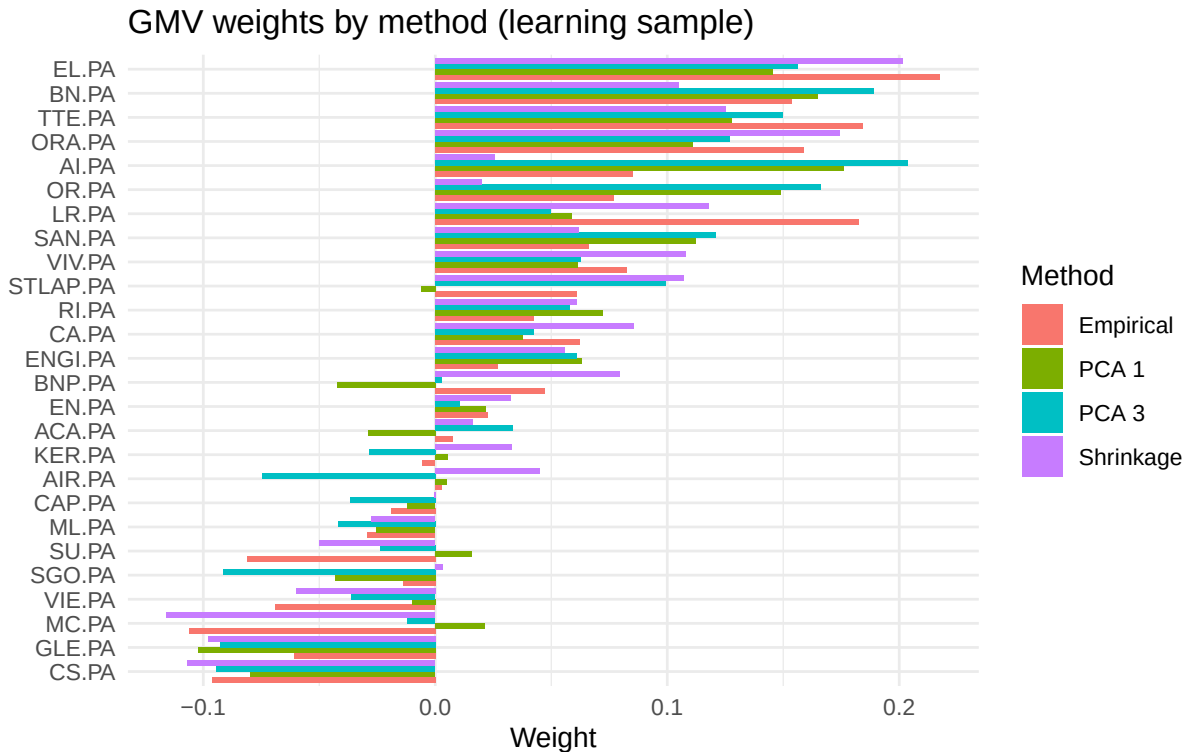
Asset	Weight
EL.PA	0.2014
ORA.PA	0.1743
TTE.PA	0.1252
LR.PA	0.1179
VIV.PA	0.1079
STLAP.PA	0.1073
BN.PA	0.1051
CA.PA	0.0857
BNP.PA	0.0797
SAN.PA	0.0619

```

weights_long <- tibble(
  Asset = assets,
  Empirical = omega_emp,
  `PCA 1` = omega_pca1,
  `PCA 3` = omega_pca3,
  Shrinkage = omega_shr
) %>%
pivot_longer(-Asset, names_to = "Method", values_to = "Weight")

ggplot(weights_long, aes(x = reorder(Asset, Weight), y = Weight, fill = Method)) +
  geom_col(position = "dodge") +
  coord_flip() +
  theme_minimal() +
  labs(title = "GMV weights by method (learning sample)", x = "", y = "Weight")

```



Empirically, the comparison between one-factor and three-factor GMV portfolios illustrates the classical bias–variance trade-off. The one-factor model produces smoother and more stable allocations, suggesting that a large fraction of the relevant covariance structure is captured by a single systematic component. Introducing additional factors increases flexibility but may reintroduce estimation noise when the sample size is limited. This result highlights that higher model complexity does not necessarily translate into better out-of-sample performance in the GMV context.

While volatility is the primary optimization criterion in the GMV framework, it is not sufficient to fully characterize portfolio quality. Portfolios achieving low in-sample variance may do so through

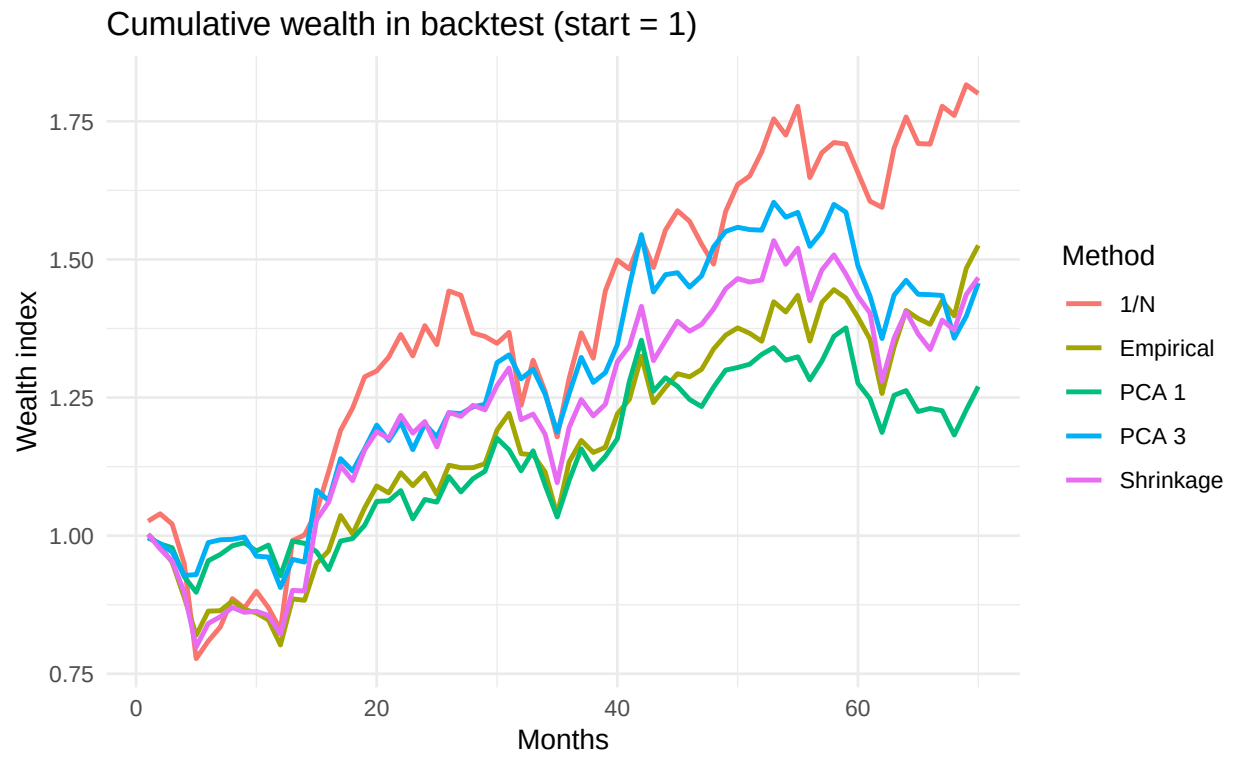
extreme and economically implausible allocations. To address this issue, volatility analysis is complemented with concentration diagnostics, such as the Herfindahl–Hirschman index and the maximum individual asset weight. These measures allow us to distinguish genuine risk reduction from artificial variance minimization driven by highly concentrated positions. ### Cumulative wealth in backtest

This plot does not replace risk evaluation, but it helps communicate the realized path of each strategy on the backtest period.

```
ret_emp <- as.matrix(backtest) %*% omega_emp
ret_pca1 <- as.matrix(backtest) %*% omega_pca1
ret_pca3 <- as.matrix(backtest) %*% omega_pca3
ret_shr <- as.matrix(backtest) %*% omega_shr
ret_1N <- as.matrix(backtest) %*% omega_1N

wealth <- tibble(
  t = 1:nrow(backtest),
  Empirical = cumprod(1 + ret_emp),
  `PCA 1` = cumprod(1 + ret_pca1),
  `PCA 3` = cumprod(1 + ret_pca3),
  Shrinkage = cumprod(1 + ret_shr),
  `1/N` = cumprod(1 + ret_1N)
) %>%
pivot_longer(-t, names_to = "Method", values_to = "Wealth")

ggplot(wealth, aes(x = t, y = Wealth, color = Method)) +
  geom_line(linewidth = 0.9) +
  theme_minimal() +
  labs(title = "Cumulative wealth in backtest (start = 1)", x = "Months", y = "Wealth index")
```



1.7 Conclusion

This report extends the workshop by adding a full robustness comparison of GMV allocations under multiple covariance estimators and by improving result communication through concentration diagnostics and weight-by-asset tables. In the Markowitz GMV setting, estimation risk is critical because portfolio weights depend on $\widehat{\Sigma}^{-1}$. PCA filtering and covariance shrinkage represent two standard approaches to reduce estimation noise. The empirical evaluation focuses on the ex-ante vs ex-post volatility gap, weight concentration, and stability across methods. These elements provide an academic-style diagnostic of whether an apparently “optimal” in-sample allocation remains coherent out of sample.

Overall, this empirical analysis confirms that estimation risk is the dominant practical issue in the Global Minimum Variance framework. Regularization techniques such as PCA factor filtering and covariance shrinkage significantly improve the stability of portfolio weights and reduce the discrepancy between theoretical and realized volatility. The results emphasize the importance of combining volatility measures with concentration diagnostics when assessing optimized portfolios and illustrate the trade-off between theoretical optimality and empirical robustness in portfolio construction.

2 Worskhop 2 — Review with Dynamic Allocation and Machine Learning views

2.1 Introduction

This project builds on a baseline portfolio allocation framework provided during the course, whose objective was to introduce the Markowitz mean–variance model and a first illustration of the Black–Litterman (BL) approach. The original code proposes a simple and pedagogical pipeline: monthly returns are computed from a small universe of assets, expected returns and the covariance matrix are estimated on a learning sample, and portfolio weights are obtained through a closed-form mean–variance solution. An illustrative Black–Litterman extension is then introduced by adding subjective views in the form of asset ratings.

While this baseline is well suited for understanding the mechanics of portfolio optimization, it also highlights several limitations that naturally arise when moving from a classroom setting to a more realistic asset management perspective. In particular, the reliance on sample estimates for both expected returns and the covariance matrix exposes the portfolio to significant estimation risk. Because optimal weights depend on the inverse of the covariance matrix, small estimation errors can be strongly amplified, leading to unstable allocations and poor out-of-sample performance. In addition, the baseline implementation relies on a static train–test split and on hand-crafted views, which limits its empirical robustness and its economic interpretation.

The objective of this project is therefore not to challenge the Markowitz or Black–Litterman frameworks themselves, but to propose a set of practical and methodologically coherent improvements to the baseline code. These improvements are designed to remain fully consistent with the spirit of the original approach, while making it more robust and closer to how portfolio allocation problems are addressed in practice.

Concretely, the contributions of this project are fourfold.

First, we extend the data setting by moving from a short monthly sample to a longer daily dataset, starting in 2012. This choice allows us to exploit a richer information set and to reduce the dependence of the results on a specific market regime. Portfolio decisions are still taken at a monthly frequency, which strikes a balance between responsiveness and noise.

Second, we improve risk estimation by relying on covariance shrinkage techniques rather than on the raw sample covariance matrix. Shrinkage explicitly addresses the bias–variance trade-off in covariance estimation and is known to stabilize portfolio weights in finite samples.

Third, the evaluation framework is made more realistic by adopting a rolling, walk-forward rebalancing scheme. Expected returns and covariance matrices are re-estimated over moving windows, and portfolios are rebalanced monthly, which allows for a genuine out-of-sample assessment of performance and stability.

Finally, we revisit the Black–Litterman extension by replacing subjective, ad hoc views with systematic views derived from a machine learning model (XGBoost). Rather than using the model as a direct return forecaster, its predictions are transformed into ranking-based views that generate small, controlled tilts around a Markowitz prior. Confidence in these views is deliberately kept low through the specification of the BL uncertainty matrix, so that the model acts as an information overlay rather than as a dominant signal.

Overall, the project aims to illustrate how the classical Markowitz and Black–Litterman frameworks can be adapted and combined with modern estimation techniques and machine learning tools, while preserving interpretability, robustness, and economic coherence.

2.2 Data and Preprocessing

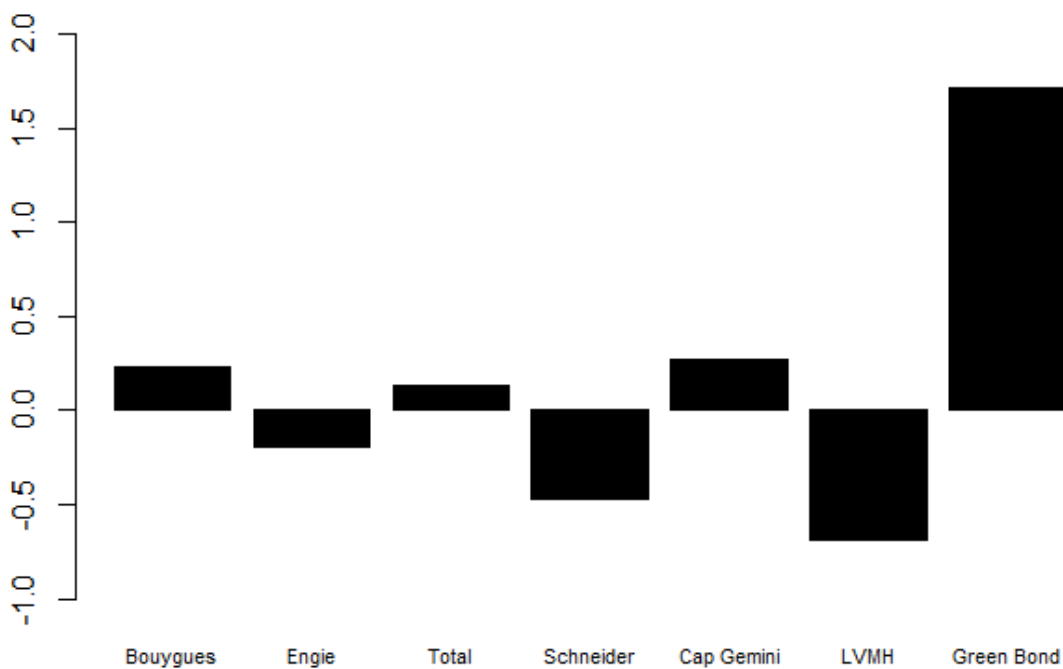
The baseline included a “Green Bond” proxy (BGRN). In practice, this ticker does not provide a sufficiently long history (it starts around 2018), which prevents us from extending the sample backward without introducing an unbalanced panel. Since the objective of this project is to increase the information set and reduce regime-specific biases, we remove BGRN and focus on a consistent universe of six French large-cap equities:

- EN.PA (Bouygues), ENGI.PA (Engie), TTE.PA (TotalEnergies), SU.PA (Schneider Electric), CAP.PA (Capgemini), MC.PA (LVMH)

This choice ensures that the cross-section is available on the full sample period and avoids artificial distortions due to missing data.

2.2.1 Frequency and sample period

A key limitation of the baseline approach is that it relies on monthly returns over 2018–2025, which can make results overly dependent on a specific market regime (momentum effects, post-COVID rebound, etc.). To mitigate this, we extend the sample back to 2012 and work at a daily frequency. This increases the number of observations used to estimate risk and return inputs. From experience, we observe that a large share of the performance of the original model was driven by a short position in LVMH, whose price declined significantly during part of the sample period.



The baseline model also exhibits a high degree of instability. In practice, minor modifications to the train–test split — for instance shifting the split date by a single month — result in substantial changes in out-of-sample performance.

The trade-off is that daily data contains more short-term noise, but empirically this is compensated by the gain in statistical robustness when estimating parameters, especially the covariance matrix.

2.2.2 Return construction: log returns, then simple returns

Daily returns are first computed as log returns:

$$r_t^{\log} = \log \left(\frac{P_t}{P_{t-1}} \right),$$

which are numerically stable and additive over time. For portfolio aggregation and backtesting, we then convert log returns to arithmetic (simple) returns:

$$r_t = \exp(r_t^{\log}) - 1.$$

This ensures that portfolio returns are computed in the standard way as a weighted sum of simple asset returns.

2.2.3 Data alignment

Daily log returns are reshaped into a wide matrix (one column per asset, one row per trading day). We keep a clean and consistent sample by forward-aligning assets and dropping incomplete dates so that all portfolios are built on the same set of observations

```
symbols <- c("EN.PA", "ENGI.PA", "TTE.PA", "SU.PA", "CAP.PA", "MC.PA")

prices <- tq_get(symbols, get = "stock.prices",
                from = "2012-11-27", to = "2025-09-30") %>%
  select(symbol, date, adjusted) %>%
  arrange(symbol, date)

daily_ret <- prices %>%
  group_by(symbol) %>%
  mutate(ret_log = log(adjusted / lag(adjusted))) %>%
  ungroup() %>%
  select(date, symbol, ret_log) %>%
  drop_na()

R_tbl <- daily_ret %>%
  pivot_wider(names_from = symbol, values_from = ret_log) %>%
  arrange(date) %>%
  drop_na()

R_log_xts <- xts(as.matrix(select(R_tbl, -date)), order.by = R_tbl$date)
colnames(R_log_xts) <- symbols

# IMPORTANT: convert log returns to arithmetic/simple returns
R_xts <- exp(R_log_xts) - 1
```

2.3 Rebalancing framework and core implementation helpers

A key limitation of the baseline model is that it effectively produces a *static* allocation: weights are estimated once on a learning sample and then held constant over the backtest period. In practice, portfolio allocation is a dynamic process: risk and return estimates are updated over time and weights are re-optimized at regular rebalancing dates. This project therefore introduces a monthly rebalancing schedule on top of daily data, which allows us to combine a rich information set (daily observations) with a realistic decision frequency (monthly trades).

2.3.1 Monthly rebalancing dates from daily returns

We work with daily returns R_t , but portfolio weights are only updated at month-end. This is implemented using `endpoints()` from `xts`, which returns the index positions corresponding to month boundaries. We also apply basic safety checks to avoid invalid indices at the beginning and end of the sample.

Conceptually, at each rebalancing date t , we: 1. estimate model inputs (μ_t, Σ_t) from a rolling historical window, 2. compute optimal weights w_t , 3. apply w_t to daily returns until the next month-end.

2.3.2 Performance metrics (annualized)

To evaluate strategies consistently, we compute standard annualized performance measures from the daily portfolio return series:

- annualized return (geometric),
- annualized volatility,
- Sharpe ratio (with $r_f = 0$ in this project),
- maximum drawdown.

Annualization is based on `perio = 252`, consistent with trading days in a year.

2.3.3 Risk estimation: shrinkage covariance

A central improvement relative to the baseline is the estimation of the covariance matrix. The sample covariance matrix is known to be noisy in finite samples, and optimization tends to amplify this noise through matrix inversion. To mitigate this issue, we replace the raw empirical covariance with a shrinkage estimator (Ledoit–Wolf style via `cov.shrink`). The resulting covariance matrix is better conditioned numerically and typically produces more stable weights.

In the implementation, the covariance is estimated on a rolling window (defined later) and then annualized.

2.3.4 Markowitz optimizer under realistic constraints

Finally, portfolio weights are computed by solving a constrained mean–variance optimization problem using quadratic programming (`quadprog`). The optimization enforces:

- a full-investment constraint: $\sum_i w_i = 1$,
- lower and upper bounds: $w_{\min} \leq w_i \leq w_{\max}$.

These bounds are essential in practice: they prevent extreme allocations driven by estimation noise and allow us to control leverage and concentration. In the final calibrated model, we use a long-only constraint (no shorts) and cap maximum exposure per asset, as discussed later in the robustness section.

```
rebal_dates <- endpoints(R_xts, on = "months")
rebal_dates <- rebal_dates[rebal_dates > 0]
rebal_dates <- rebal_dates[rebal_dates < nrow(R_xts)]
```

```

perio <- 252

perf_metrics <- function(port_ret_xts, perio = 252) {
  port_ret_xts <- port_ret_xts[!is.na(port_ret_xts)]
  ann_return <- (prod(1 + port_ret_xts)^(perio / nrow(port_ret_xts)) - 1)
  ann_vol <- sd(port_ret_xts) * sqrt(perio)
  sharpe <- as.numeric(ann_return / ann_vol)
  mdd <- maxDrawdown(port_ret_xts)
  list(ann_return = ann_return, ann_vol = ann_vol, sharpe = sharpe, mdd = mdd)
}

cov_shrink_ann <- function(R_window, perio = 252) {
  corpcor::cov.shrink(R_window, verbose = FALSE) * perio
}

equilibrium_pi <- function(Sigma_ann, w_mkt, delta = 3) {
  as.vector(delta * Sigma_ann %*% w_mkt)
}

markowitz_mv_weights_bounds <- function(Sigma_ann, mu_ann, lambda,
                                         w_min, w_max) {
  n <- length(mu_ann)
  Dmat <- Sigma_ann
  dvec <- lambda * mu_ann

  Amat <- cbind(rep(1, n), diag(n), -diag(n))
  bvec <- c(1, rep(w_min, n), rep(-w_max, n))

  sol <- solve.QP(Dmat, dvec, Amat, bvec, meq = 1)
  w <- sol$solution
  w[abs(w) < 1e-10] <- 0
  w
}

```

2.4 Benchmark strategy: equal-weight (buy-and-hold with monthly rebalancing)

Before evaluating more sophisticated allocation rules, it is essential to define a simple and economically meaningful benchmark. In line with the empirical portfolio choice literature, we use an equal-weighted ($1/N$) portfolio as a baseline reference strategy.

The equal-weight portfolio is often considered a strong benchmark in the presence of estimation risk. As shown by DeMiguel, Garlappi, and Uppal (2009), naive diversification can perform competitively relative to optimized portfolios when parameter estimates are noisy. Using such a benchmark allows us to assess whether the additional complexity of optimization and model tuning is justified in terms of out-of-sample performance.

In our implementation, the equal-weight strategy is constructed dynamically using the same rebal-

ancing schedule as the optimized portfolios. At each month-end rebalancing date, portfolio weights are reset to $w_i = 1/N$, and these weights are then applied to daily returns until the next rebalancing date. This ensures that the benchmark is fully comparable to the Markowitz and Black–Litterman strategies in terms of timing, data frequency, and transaction structure.

The resulting equal-weight portfolio therefore serves as a coherent “buy-and-hold with rebalancing” benchmark, against which we evaluate both risk-adjusted performance and drawdown behavior of the optimized strategies.

```
n <- ncol(R_xts)
w_eq <- rep(1/n, n)

port_ret_EW <- rep(NA_real_, nrow(R_xts))
W_hist_EW <- list()

for (k in seq_along(rebal_dates)) {
  t_idx <- rebal_dates[k]
  W_hist_EW[[as.character(index(R_xts)[t_idx])]] <- w_eq

  next_idx <- if (k < length(rebal_dates)) rebal_dates[k + 1]
  else nrow(R_xts)
  next_idx <- min(next_idx, nrow(R_xts))
  if ((t_idx + 1) > next_idx) next

  apply_range <- (t_idx + 1):next_idx
  port_ret_EW[apply_range] <- as.vector(R_xts[apply_range, ] %*% w_eq)
}

port_ret_EW_xts <- xts(port_ret_EW, order.by = index(R_xts))
port_ret_EW_xts <- port_ret_EW_xts[!is.na(port_ret_EW_xts)]
colnames(port_ret_EW_xts) <- "EW"
```

2.5 Markowitz benchmark with shrinkage, rolling estimation, and stabilized weights

The next step is to build a Markowitz benchmark that is not only theoretically correct, but also empirically robust. The baseline mean–variance allocation is extremely sensitive to the estimation of inputs, and early tests confirmed that performance was highly regime-dependent: small changes in the sample or in the optimization constraints could generate large swings in weights and out-of-sample results.

To address this, we progressively tuned the model with the objective of stabilizing allocations over time while keeping an economically interpretable structure. The final specification relies on four key design choices: (i) rolling estimation windows with monthly rebalancing, (ii) shrinkage covariance estimation, (iii) a shrunk expected return model combining historical information with a prior, and (iv) weight stabilization through constraints and smoothing.

2.5.1 Rolling estimation windows: separating risk and return horizons

We estimate risk and expected returns on different horizons. Covariance estimation requires more data to be stable, while expected returns should remain responsive to recent market dynamics. We therefore use:

- a 3-year window for the covariance matrix Σ_t (risk),
- a 1-year window for expected returns μ_t (return).

This separation is important in practice. In particular, after large momentum phases (e.g., 2020–2021), a long return window can lead to “stale” return estimates that react too slowly when regimes change (e.g., 2022). A shorter return window improves the model’s ability to adapt at rebalancing dates.

2.5.2 Risk model: shrinkage covariance

Instead of the raw empirical covariance, we estimate Σ_t using covariance shrinkage and annualize it using 252 trading days. The objective is to reduce estimation noise and improve numerical conditioning, which directly stabilizes portfolio weights given that mean–variance optimization requires matrix inversion.

2.5.3 Expected returns model: shrinkage toward a prior

The main source of instability in mean–variance optimization is usually the expected return vector. A pure historical mean is noisy and can generate extreme and fragile allocations. We therefore define expected returns as a convex combination:

$$\mu_t^{\text{shr}} = \alpha \hat{\mu}_t + (1 - \alpha) \mu_{0,t},$$

where: - $\hat{\mu}_t$ is the annualized historical mean over the 1-year window, - $\mu_{0,t}$ is a prior (either a neutral zero prior or an equilibrium prior), - $\alpha \in [0, 1]$ controls the intensity of the shrinkage.

In the final model we use an equilibrium prior $\mu_{0,t} = \delta \Sigma_t w^{mkt}$, with w^{mkt} taken as equal weights and δ interpreted as a risk-aversion scaling parameter. This acts as a structural anchor for expected returns and improves out-of-sample stability.

2.5.4 Constraints: removing shorts and controlling concentration

Initial versions allowed short positions. Empirically, we found that allowing shorts made the strategy more unstable and riskier without delivering consistent out-of-sample benefits. This is consistent with the idea that optimized short exposures can easily reflect estimation noise rather than true economic signals.

As a result, the final benchmark imposes a long-only constraint and caps individual weights:

$$0 \leq w_i \leq w_{\max}, \quad \sum_i w_i = 1.$$

This reduces extreme allocations, improves interpretability, and prevents performance from being driven by a small number of aggressive positions.

2.5.5 Turnover control: exponential smoothing of weights

Finally, even with shrinkage and constraints, monthly re-optimization can generate high turnover if weights jump from one optimum to another. To avoid unstable trading behavior and frequent boundary-hitting, we apply exponential smoothing:

$$w_t = (1 - \gamma) w_{t-1} + \gamma w_t^{raw},$$

where w_t^{raw} is the optimizer output and $\gamma \in (0, 1]$ controls how fast the portfolio moves toward the new optimum. With $\gamma = 0.30$, the model reacts to new information while keeping weight paths relatively smooth over time.

Overall, this tuned Markowitz specification provides a robust and credible benchmark. It substantially improves stability relative to the baseline implementation and serves as the natural prior portfolio before introducing Black–Litterman views.

```

window_years_S <- 3
window_days_S  <- perio * window_years_S

window_years_mu <- 1
window_days_mu  <- perio * window_years_mu

alpha          <- 0.3
rf             <- 0
lambda_mv      <- 1
w_min          <- 0
w_max          <- 0.85

use_mu0_pi     <- TRUE
delta          <- 3
w_mkt          <- rep(1/n, n)

port_ret_MK    <- rep(NA_real_, nrow(R_xts))
W_hist_MK      <- list()

gamma          <- 0.30
w_prev         <- rep(1/n, n)

for (k in seq_along(rebal_dates)) {

  t_idx <- rebal_dates[k]
  if (t_idx <= max(window_days_S, window_days_mu)) next

```

```

R_S <- R_xts[(t_idx - window_days_S + 1):t_idx, ]
Sigma_ann <- cov_shrink_ann(coredata(R_S), perio = perio)

R_mu <- R_xts[(t_idx - window_days_mu + 1):t_idx, ]
mu_hat <- colMeans(coredata(R_mu)) * perio - rf

mu0 <- if (use_mu0_pi) equilibrium_pi(Sigma_ann, w_mkt = w_mkt, delta = delta)
else rep(0, n)

mu_shr <- alpha * mu_hat + (1 - alpha) * mu0

w_raw <- markowitz_mv_weights_bounds(Sigma_ann, mu_ann = mu_shr,
                                     lambda = lambda_mv,
                                     w_min = w_min, w_max = w_max)

w <- (1 - gamma) * w_prev + gamma * w_raw
w <- w / sum(w)
w_prev <- w

W_hist_MK[[as.character(index(R_xts)[t_idx])]] <- w

next_idx <- if (k < length(rebal_dates)) rebal_dates[k + 1] else nrow(R_xts)
next_idx <- min(next_idx, nrow(R_xts))
if ((t_idx + 1) > next_idx) next

apply_range <- (t_idx + 1):next_idx
port_ret_MK[apply_range] <- as.vector(R_xts[apply_range, ] %*% w)
}

port_ret_MK_xts <- xts(port_ret_MK, order.by = index(R_xts))
port_ret_MK_xts <- port_ret_MK_xts[!is.na(port_ret_MK_xts)]
colnames(port_ret_MK_xts) <- "Markowitz_muShrink"

W_MK_mat <- do.call(rbind, W_hist_MK)
colnames(W_MK_mat) <- colnames(R_xts)
W_MK_df <- data.frame(date = as.Date(rownames(W_MK_mat)),
                      W_MK_mat, row.names = NULL)

```

2.6 Results (Part I): equal-weight benchmark vs tuned Markowitz strategy

In order to ensure a fair comparison between strategies, all performance metrics are computed on a common sample, defined as the intersection of dates for which both portfolio return series are available. This adjustment is necessary because the Markowitz strategy requires an initial estimation window (in particular for the covariance matrix), which mechanically delays the start of the backtest. Comparing strategies outside a common sample would therefore introduce a bias.

Over the common period ranging from December 2015 to September 2025 (a total of 2,517 daily observations), the tuned Markowitz strategy clearly dominates the equal-weight benchmark in terms of risk-adjusted performance.

The equal-weight (1/N) portfolio delivers an annualized return of 11.35%, with an annualized volatility of 19.69%, corresponding to a Sharpe ratio of 0.58. The maximum drawdown reaches 41.1%, reflecting the full exposure of the benchmark to adverse market phases.

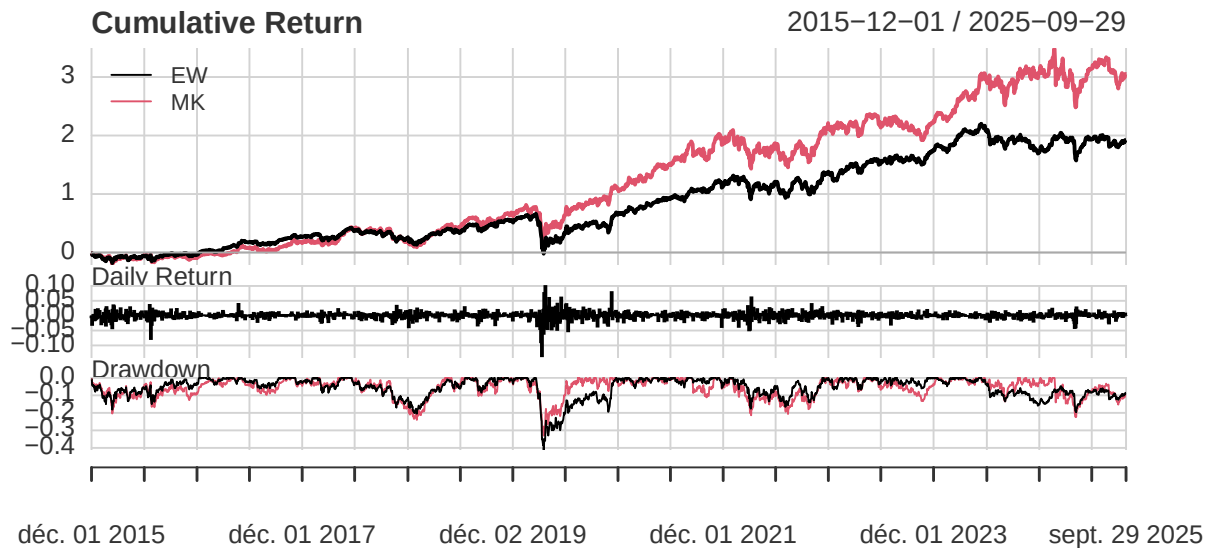
By contrast, the Markowitz strategy with shrinked expected returns, long-only constraints, and weight smoothing achieves a higher annualized return of 15.02%, at the cost of a moderately higher volatility of 23.0%. Importantly, the improvement in returns more than compensates for the additional risk, resulting in a higher Sharpe ratio of 0.65. Moreover, the maximum drawdown is substantially reduced to 32.96%, indicating improved downside protection relative to the benchmark.

Overall, these results suggest that the tuning choices introduced in the Markowitz framework — in particular shrinkage, rolling estimation, constraints, and smoothing — significantly enhance the stability and efficiency of the allocation. The strategy not only improves average performance but also delivers a more favorable drawdown profile, which is essential from a portfolio management perspective.

```
both <- merge(port_ret_EW_xts, port_ret_MK_xts, join = "inner")
colnames(both) <- c("EW", "MK")

charts.PerformanceSummary(both,
                           main = "EW vs Markowitz (common sample)")
```

EW vs Markowitz (common sample)



```
m_EW <- perf_metrics(both$EW, perio)
m_MK <- perf_metrics(both$MK, perio)

cat("Common start:", as.character(start(both)), "\n")
```

```
## Common start: 2015-12-01
```

```
cat("Common end :", as.character(end(both)), "\n")
```

```
## Common end : 2025-09-29
```

```
cat("N obs      :", nrow(both), "\n\n")
```

```
## N obs      : 2517
```

```
cat("=== Equal-Weight (1/N) Benchmark ===\n")
```

```
## === Equal-Weight (1/N) Benchmark ===
```

```
cat("Annualized return:", round(100 * m_EW$ann_return, 2), "%\n")
```

```
## Annualized return: 11.35 %
```

```
cat("Annualized vol    :", round(100 * m_EW$ann_vol, 2), "%\n")
```

```
## Annualized vol    : 19.69 %
```

```
cat("Sharpe (rf=0)     :", round(m_EW$sharpe, 2), "%\n")
```

```
## Sharpe (rf=0)     : 0.58
```

```
cat("Max Drawdown      :", round(100 * m_EW$mdd, 2), "%\n\n")
```

```
## Max Drawdown      : 41.1 %
```

```
cat("=== Markowitz (mu shrink, long-only, smoothed) ===\n")
```

```
## === Markowitz (mu shrink, long-only, smoothed) ===
```

```
cat("Annualized return:", round(100 * m_MK$ann_return, 2), "%\n")
```

```
## Annualized return: 15.02 %
```

```
cat("Annualized vol    :", round(100 * m_MK$ann_vol, 2), "%\n")
```

```
## Annualized vol    : 23 %
```

```
cat("Sharpe (rf=0)     :", round(m_MK$sharpe, 2), "%\n")
```

```
## Sharpe (rf=0)     : 0.65
```

```
cat("Max Drawdown      :", round(100 * m_MK$mdd, 2), "%\n\n")
```

```
## Max Drawdown      : 32.96 %
```

2.7 Backtest diagnostics and robustness analysis

Beyond aggregate performance metrics, it is essential to analyze how the Markowitz strategy behaves over time in terms of weight stability, turnover, and robustness across market regimes. This section provides a detailed diagnostic of the backtest results on the common sample (December 2015 – September 2025).

2.7.1 Weight stability and exposure diagnostics

On the common sample of 2,517 daily observations, the tuned Markowitz strategy exhibits stable and economically reasonable portfolio characteristics.

The average maximum absolute weight is approximately 0.65 (median 0.68), indicating that the portfolio remains moderately concentrated without collapsing into single-asset bets. Importantly, the gross leverage is exactly 1 on average, reflecting the long-only constraint and confirming the absence of hidden leverage. Net exposure is also very close to 1 at all times, as expected under the full-investment constraint.

The imposed bounds are rarely binding. The upper bound $w_{\max}$ is never hit, while the lower bound $w_{\min} = 0$ is reached in only 5.9% of months. This suggests that the constraint set primarily acts as a safeguard rather than as an active driver of allocations.

Finally, portfolio turnover remains low, with an average monthly turnover of 0.09 (median 0.08). This confirms that the exponential smoothing mechanism effectively dampens abrupt changes in weights and prevents excessive trading.

2.7.2 Average portfolio composition

An analysis of average absolute weights provides additional insight into the structure of the allocation. Over the full sample, the largest average exposures are observed in Schneider Electric (SU.PA) and Capgemini (CAP.PA), followed by Bouygues (EN.PA) and LVMH (MC.PA). By contrast, TotalEnergies (TTE.PA) and Engie (ENGI.PA) receive smaller average allocations.

These results are consistent with the cross-sectional characteristics of the assets over the period and confirm that the strategy does not rely on extreme or unstable positions to generate performance.

2.7.3 Performance across sub-periods and market regimes

To further assess robustness, performance is evaluated across several sub-periods corresponding to different market regimes.

During 2016–2018, both strategies deliver modest returns, with the equal-weight portfolio slightly outperforming the Markowitz strategy. This period illustrates that the optimized strategy does not mechanically dominate in all environments.

In the 2019–early 2020 phase, the Markowitz strategy strongly outperforms, achieving a Sharpe ratio of approximately 0.65 compared to 0.05 for the equal-weight benchmark, highlighting its ability to adapt during turbulent conditions.

The 2020–2021 period corresponds to a strong recovery and momentum phase. Both strategies perform very well, but the Markowitz allocation delivers higher returns and a higher Sharpe ratio (2.26 vs 1.94), confirming that the model is able to capture favorable trends without excessive risk-taking.

By contrast, the 2022–2023 period is more challenging. While the equal-weight portfolio maintains moderate performance, the Markowitz strategy underperforms, reflecting the regime-dependent nature of optimized allocations in a context of sharp factor reversals and increased uncertainty.

Finally, in the 2024–2025 period, the Markowitz strategy again dominates in risk-adjusted terms, achieving higher returns and a Sharpe ratio of 0.50 versus 0.18 for the benchmark.

Overall, this sub-period analysis confirms that the tuned Markowitz strategy is not immune to regime shifts, but it delivers more stable performance across heterogeneous market environments than the baseline implementation. Importantly, gains are not driven by a single favorable episode but arise from a combination of controlled risk-taking, stable weights, and disciplined rebalancing.

```
cat("\n=== COMMON SAMPLE ===\n")

##
## === COMMON SAMPLE ===

cat("Start:", as.character(start(both)),
    " End:", as.character(end(both)),
    " N obs:", nrow(both), "\n")

## Start: 2015-12-01 End: 2025-09-29 N obs: 2517

W_only <- as.matrix(W_MK_df %>% select(-date))
colnames(W_only) <- colnames(R_xts)
n_reb <- nrow(W_only)

max_abs_w <- apply(abs(W_only), 1, max)
gross_lev_series <- rowSums(abs(W_only))
net_series <- rowSums(W_only)

tol <- 1e-6
hit_max_any <- apply(W_only, 1, function(x) any(abs(x - w_max) < tol))
hit_min_any <- apply(W_only, 1, function(x) any(abs(x - w_min) < tol))

turnover <- rep(NA_real_, n_reb)
if (n_reb >= 2) {
  turnover[2:n_reb] <- 0.5 * rowSums(abs(W_only[2:n_reb,
                                                    , drop=FALSE] - W_only[1:(n_reb-1),
                                                    , drop=FALSE]))
}

cat("\n=== WEIGHTS DIAGNOSTICS (Markowitz) ===\n")

##
## === WEIGHTS DIAGNOSTICS (Markowitz) ===

cat("Avg max |w|      :", round(mean(max_abs_w), 3),
    " (median:", round(median(max_abs_w), 3), ")\n")
```

```
## Avg max |w| : 0.651 (median: 0.68 )
```

```
cat("Avg gross leverage :", round(mean(gross_lev_series), 2),  
    " (median:", round(median(gross_lev_series), 2), ")\n")
```

```
## Avg gross leverage : 1 (median: 1 )
```

```
cat("Avg net exposure :", round(mean(net_series), 2), " (should be ~1)\n")
```

```
## Avg net exposure : 1 (should be ~1)
```

```
cat("% months hitting w_max:", round(100 * mean(hit_max_any), 1), "%\n")
```

```
## % months hitting w_max: 0 %
```

```
cat("% months hitting w_min:", round(100 * mean(hit_min_any), 1), "%\n")
```

```
## % months hitting w_min: 5.9 %
```

```
cat("Avg monthly turnover :", round(mean(turnover, na.rm=TRUE), 2),  
    " (median:", round(median(turnover, na.rm=TRUE), 2), ")\n")
```

```
## Avg monthly turnover : 0.09 (median: 0.08 )
```

```
avg_w      <- colMeans(W_only)  
avg_abs_w  <- colMeans(abs(W_only))  
diag_tab <- data.frame(asset = names(avg_w),  
                       avg_w = as.numeric(avg_w),  
                       avg_abs_w = as.numeric(avg_abs_w)) %>%  
  arrange(desc(avg_abs_w))  
  
cat("\nTop exposures by avg |w|:\n")
```

```
##
```

```
## Top exposures by avg |w|:
```

```
print(head(diag_tab, 6))
```

```
##      asset      avg_w avg_abs_w  
## 1  SU.PA 0.30432772 0.30432772  
## 2  CAP.PA 0.28430798 0.28430798  
## 3  EN.PA 0.15445744 0.15445744  
## 4  MC.PA 0.13733735 0.13733735  
## 5  TTE.PA 0.08508150 0.08508150  
## 6  ENGI.PA 0.03448802 0.03448802
```

```

perf_one <- function(x, perio=252) {
  x <- x[!is.na(x)]
  ann_return <- (prod(1 + x)^(perio / length(x)) - 1)
  ann_vol <- sd(x) * sqrt(perio)
  sharpe <- as.numeric(ann_return / ann_vol)
  mdd <- maxDrawdown(x)
  c(ann_return=ann_return, ann_vol=ann_vol, sharpe=sharpe, mdd=mdd)
}

cuts <- as.Date(c("2016-01-01", "2019-01-01", "2020-04-01", "2022-01-01", "2024-01-01"))
bounds <- c(as.Date(start(both)), cuts, as.Date(end(both)) + 1)

res <- list()
for (i in 1:(length(bounds)-1)) {
  a <- bounds[i]
  b <- bounds[i+1] - 1
  seg <- both[paste0(a, "/", b)]
  if (nrow(seg) < 100) next

  ew <- perf_one(seg$EW, perio)
  mk <- perf_one(seg$MK, perio)

  res[[length(res)+1]] <- data.frame(
    period = paste0(a, " / ", b),
    EW_return = ew["ann_return"], EW_vol = ew["ann_vol"],
    EW_sharpe = ew["sharpe"], EW_mdd = ew["mdd"],
    MK_return = mk["ann_return"], MK_vol = mk["ann_vol"],
    MK_sharpe = mk["sharpe"], MK_mdd = mk["mdd"]
  )
}

rob_tab <- do.call(rbind, res)

```

2.8 Appendix (Python): XGBoost pipeline and anti-bias design

The machine learning component used to generate Black–Litterman views is implemented in Python and follows a deliberately structured and conservative pipeline. Although the full code is not embedded in this R Markdown document, we summarize here the key methodological steps that are essential for understanding the role and limitations of the model.

2.8.1 Step 1 – Construction of the base dataset

The initial dataset is built using daily price data for the asset universe, from which daily log-returns are computed. These daily series are then used to construct features, while the prediction target is defined at a monthly frequency, consistent with the portfolio rebalancing schedule.

For each asset and each month-end trading date t , the target corresponds to the forward one-month return from t to $t + 1$. All features used at date t rely exclusively on information available up to that date, ensuring a strict no-look-ahead construction.

2.8.2 Step 2 – Feature engineering and macro enrichment

A first set of features is constructed at the asset level, primarily capturing standard financial effects such as short- and medium-term momentum, realized volatility, cumulative returns, drawdowns, and standardized returns. These features are computed using rolling windows on daily data and then sampled at month-end.

In a second step, the dataset is enriched with macro-financial features derived from market indices and prices (equity indices, oil, and foreign exchange rates). Macro features include rolling returns, realized volatility measures, and volatility ratios, all aligned to month-end trading dates and forward-filled where necessary to avoid calendar mismatches.

This two-layer feature design allows the model to exploit both asset-specific dynamics and broader market conditions, while remaining based on economically interpretable variables.

2.8.3 Step 3 – Feature selection via Elastic Net

Given the potentially large number of engineered features, an explicit feature selection stage is introduced prior to training the XGBoost model. Feature selection is performed using an Elastic Net regression, estimated on an early subsample of the data and evaluated through time-series cross-validation.

The Elastic Net combines ℓ_1 and ℓ_2 regularization, allowing the model to: - shrink noisy coefficients, - discard irrelevant features, - and retain a parsimonious set of predictors.

Importantly, this selection step is carried out strictly out-of-sample with respect to the subsequent XGBoost estimation. Once selected, the feature set is frozen and applied to the full dataset, preventing any form of information leakage.

2.8.4 Step 4 – XGBoost estimation with walk-forward optimization

The final predictive model is a gradient boosted tree regressor (XGBoost) estimated in a walk-forward framework. At each monthly rebalancing date:

- the training sample includes only observations dated strictly before the prediction date,
- a validation subset is extracted from the most recent months of the training period,
- hyperparameters are optimized using Bayesian optimization (Optuna), with early stopping to limit overfitting,
- predictions are generated for all assets at the current date.

Hyperparameter tuning is intentionally not performed at every step but only periodically, to reduce the risk of overfitting to short-term noise. Target values are winsorized within each training window to further improve robustness.

This code is a representative snippet illustrating the Optuna-based hyperparameter optimization used in the walk-forward XGBoost pipeline:

```
def objective(trial):  
  
    params = {  
        "objective": "reg:squarederror",  
        "eval_metric": "rmse",  
        "tree_method": "hist",  
  
        "max_depth": trial.suggest_int("max_depth", 2, 5),  
        "eta": trial.suggest_float("eta", 0.01, 0.15, log=True),  
        "subsample": trial.suggest_float("subsample", 0.6, 1.0),  
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.6, 1.0),  
  
        "gamma": trial.suggest_float("gamma", 0.0, 5.0),  
        "lambda": trial.suggest_float("lambda", 0.1, 10.0, log=True),  
        "alpha": trial.suggest_float("alpha", 0.0, 5.0),  
    }  
  
    num_round = trial.suggest_int("num_boost_round", 200, 3000)  
  
    bst = xgb.train(  
        params=params,  
        dtrain=dtrain,  
        num_boost_round=num_round,  
        evals=[(dvalid, "valid")],  
        early_stopping_rounds=EARLY_STOP,  
        verbose_eval=False  
    )  
  
    pred = bst.predict(dvalid, iteration_range=(0, bst.best_iteration + 1))  
    return rmse(y_valid, pred)
```

```
study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=N_TRIALS)
```

2.9 Machine learning signals and integration into Black–Litterman

This section introduces the machine learning component used in this project and explains how predictive signals are incorporated into the portfolio allocation process. The objective is not to build a standalone machine-learning-driven strategy, but to assess whether carefully controlled predictive information can add value when embedded within a robust and economically grounded framework.

We first discuss the expected predictive power and limitations of the machine learning model, before motivating the use of the Black–Litterman framework as a disciplined way to translate noisy signals into portfolio tilts. The section then details how machine learning predictions are transformed into conservative, ranking-based views and combined with a Markowitz prior.

The python code used to implement the XGBoost pipeline and generate out-of-sample predictions is available in the project appendix.

2.9.1 Interpretation and expected predictive power

Because the entire pipeline is built under strict anti-bias constraints — walk-forward estimation, frozen feature selection, conservative tuning, and limited feature novelty — the predictive power of the XGBoost model is expected to be modest.

This outcome is not a weakness but a direct consequence of the modeling philosophy. In a realistic investment setting, genuinely out-of-sample predictive signals are rare and unstable. For this reason, the machine learning model is not used as a standalone allocation engine but as a low-intensity information signal, whose role is to provide relative ranking information within the Black–Litterman framework.

By construction, the combination of a robust Markowitz prior and carefully constrained machine learning views allows us to test whether modern predictive tools can add marginal but robust value, without reintroducing the instability and overfitting issues observed in naive ML-driven allocation strategies.

2.9.2 From predictive signals to portfolio allocation: motivation for Black–Litterman

The construction of the XGBoost model provides a set of fully out-of-sample predictions that capture cross-sectional information about future asset performance. However, translating these predictions directly into portfolio weights would be neither robust nor economically sound. Machine learning signals are noisy by nature, sensitive to regime changes, and difficult to interpret as precise expected returns.

The objective at this stage of the project is therefore not to build a standalone machine-learning-driven allocation, but to evaluate whether a carefully tuned machine learning signal can add value when embedded within a well-established portfolio framework.

The Black–Litterman (BL) model provides a natural and disciplined way to address this question. In our setting, the tuned Markowitz strategy developed in the previous sections plays the role of a structural prior, reflecting long-term risk–return trade-offs estimated from historical data and stabilized through shrinkage, constraints, and smoothing. The machine learning model, by contrast, is treated as a source of additional information, whose influence should remain limited and conditional.

Rather than interpreting XGBoost predictions as absolute return forecasts, we deliberately adopt a conservative approach. Predictions are used only to generate relative, ranking-based views, such as identifying a small subset of assets expected to outperform or underperform their peers. These views are designed to be small in magnitude and associated with low confidence, so that they induce only marginal tilts around the Markowitz allocation.

This design choice allows us to answer a clear empirical question:

Can machine learning signals, when carefully constrained and integrated through Black–Litterman, improve portfolio performance without compromising stability?

The following section details the implementation of this Black–Litterman extension.

2.9.3 Black–Litterman views: from subjective ratings to ML-based ranking views

The baseline Black–Litterman (BL) illustration provided in the course relies on hand-crafted, subjective views. Concretely, the baseline code assigns each asset a discrete “rating” (`note`) and converts this rating into an adjusted return vector Q through a simple scaling rule:

- views are absolute (each asset gets a level adjustment),
- view strength is controlled by a global parameter `theta`,
- uncertainty is set in a very simple way using a diagonal matrix (`Omega <- diag(diag(Sigma))`),
- the posterior mean is computed through a mixed estimation formula (`mu_mixed`).

This is pedagogical and easy to interpret, but it has two practical drawbacks:

- (1) views are not systematically generated (they depend on manual ratings), and
- (2) the framework does not explicitly enforce “softness” of the views: depending on `theta` and the scale of volatility, the implied adjustments can become economically large.

2.9.4 What we changed: systematic, out-of-sample views from XGBoost

Our main innovation is to replace subjective ratings with data-driven signals coming from the walk-forward XGBoost model. Predictions are imported from `xgb_predictions_walkforward.csv` and standardized into a clean table:

- `date`: prediction timestamp,
- `symbol`: asset identifier,
- `signal`: model output (`y_pred`).

Crucially, we align signals to the portfolio decision process using an as-of rule: at each rebalancing date t , we only use the latest available prediction dated $\leq t$. This prevents look-ahead and ensures the BL views are strictly out-of-sample.

2.9.5 Ranking-based views instead of level views

Another key difference is how views are formulated. In the baseline code, views are essentially *absolute* return adjustments (Q is a modified vector of expected returns). In our implementation, views are relative and ranking-based:

- we identify the top k assets by signal as “long candidates”,
- the bottom k assets as “short candidates”,
- and we create pairwise views of the form:

$$r_{\text{long}} - r_{\text{short}} = q.$$

This produces a view matrix P with $+1$ on the long asset and -1 on the short asset, and a Q vector with a small, fixed view magnitude.

This choice is intentional: it avoids relying on the absolute calibration of the model outputs and focuses on the cross-sectional ordering, which is typically the most robust part of a predictive signal.

2.9.6 Soft views: fixed small magnitude and conditional activation

To keep the integration conservative, the view magnitude is set to a small constant (`q_month = 0.005`, i.e. 0.5% per month, annualized in the code). This makes BL act as a tilt rather than a full reallocation engine.

We also implement a simple robustness filter: if the dispersion of signals is too small (or if too many signals are missing), we impose no views for that month. In that case, the posterior is exactly the prior.

2.9.7 Explicit low-confidence Omega specification

In the professor’s implementation, the uncertainty matrix is taken as a simple diagonal of the covariance matrix. In our implementation, uncertainty is explicitly scaled to ensure low confidence:

$$\Omega = \frac{1}{c} \text{diag}(P \tau \Sigma P^\top),$$

implemented through `c_omega`. With a small c (e.g., 0.05), Ω becomes large, meaning views are treated as weak information. This guarantees that the posterior mean remains close to the Markowitz prior unless signals are both available and sufficiently informative.

Overall, relative ranking views + small Q + large Ω provide a disciplined way to test whether machine learning can add marginal value, without reintroducing the instability issues typical of aggressive forecasting-based allocations.

```
library(readr)
```

```
pred_path <- "C:/Users/Lopez/OneDrive/Documents/XGB_strategy/xgb_predictions_walkforward.csv"
```

```

pred_tbl <- read_csv(pred_path, show_col_types = FALSE) %>%
  transmute(
    date = as.Date(date),
    symbol = as.character(asset),
    signal = as.numeric(y_pred)
  ) %>%
  drop_na(date, symbol, signal)

get_signal_asof <- function(pred_tbl, assets, t_date) {
  pred_tbl %>%
    filter(symbol %in% assets, date <= t_date) %>%
    group_by(symbol) %>%
    slice_max(order_by = date, n = 1, with_ties = FALSE) %>%
    ungroup() %>%
    right_join(tibble(symbol = assets), by = "symbol") %>%
    arrange(match(symbol, assets)) %>%
    pull(signal)
}

build_views_ranking <- function(signal_vec, assets,
                                top_k = 2, bot_k = 2,
                                q_month = 0.005,
                                disp_min = 1e-12) {

  names(signal_vec) <- assets
  ok <- which(!is.na(signal_vec))

  if (length(ok) < 2) {
    return(list(P = NULL, Q = NULL, have_views = FALSE))
  }

  sd_sig <- sd(signal_vec[ok])
  if (!is.finite(sd_sig) || sd_sig <= disp_min) {
    return(list(P = NULL, Q = NULL, have_views = FALSE))
  }

  ord_assets <- assets[order(signal_vec, decreasing = TRUE, na.last = NA)]

  longs <- head(ord_assets, top_k)
  shorts <- tail(ord_assets, bot_k)

  longs <- setdiff(longs, shorts)
  shorts <- setdiff(shorts, longs)

  if (length(longs) == 0 || length(shorts) == 0) {

```

```

    return(list(P = NULL, Q = NULL, have_views = FALSE))
}

pairs <- expand.grid(long = longs, short = shorts, stringsAsFactors = FALSE)
k <- nrow(pairs)
n <- length(assets)

P <- matrix(0, nrow = k, ncol = n)
colnames(P) <- assets
for (i in 1:k) {
  P[i, pairs$long[i]] <- 1
  P[i, pairs$short[i]] <- -1
}

Q <- rep(q_month * 12, k)

list(P = P, Q = Q, have_views = TRUE)
}

bl_posterior_mu <- function(mu_prior, Sigma, P, Q,
                           tau = 0.05,
                           c_omega = 0.05) {

  if (is.null(P) || is.null(Q)) {
    return(list(mu = mu_prior, used_views = FALSE))
  }

  tauSigma <- tau * Sigma
  M <- P %*% tauSigma %*% t(P)

  Omega <- (1 / c_omega) * diag(diag(M))

  A <- M + Omega
  rhs <- Q - as.vector(P %*% mu_prior)

  x <- solve(A, rhs)
  mu_bl <- as.vector(mu_prior + tauSigma %*% t(P) %*% x)

  list(mu = mu_bl, used_views = TRUE)
}

```

2.9.8 Black–Litterman backtest with machine learning views

This section presents the implementation of the Black–Litterman strategy that integrates machine learning signals into the portfolio allocation process. The objective is to assess whether XGBoost-based views can improve the performance of the stabilized Markowitz strategy developed earlier, while preserving robustness and avoiding excessive turnover or instability.

2.9.9 Black–Litterman parameterization: deliberately soft views

The Black–Litterman model is parameterized in a deliberately conservative way. The parameters are chosen so that machine learning views act as small tilts around the Markowitz prior, rather than as a dominant allocation driver.

- The scalar τ controls the uncertainty on the prior mean and is set to a low value ($\tau = 0.05$), ensuring limited sensitivity of the posterior to the views.
- The confidence parameter c_ω governs the size of the uncertainty matrix Ω . A relatively high value ($c_\omega = 0.15$) implies low confidence in the views, keeping the posterior close to the prior.
- At each rebalancing date, only the top two and bottom two assets according to the XGBoost signal are used to form views.
- View magnitudes are fixed at 0.5% per month (annualized in the code), ensuring economically small adjustments.

This design reflects the assumption that machine learning signals are informative but noisy, and should therefore only have a limited influence on portfolio decisions.

2.9.10 Monthly walk-forward Black–Litterman backtest

The Black–Litterman strategy is evaluated in a monthly walk-forward framework, fully consistent with the Markowitz backtest presented earlier. At each rebalancing date:

1. The covariance matrix is estimated over a rolling three-year window using shrinkage, ensuring stable risk estimates.
2. Expected returns are computed over a one-year window and combined with an equilibrium prior through shrinkage, yielding the Markowitz prior mean.
3. The most recent available XGBoost predictions are retrieved using an as-of rule, guaranteeing strict out-of-sample usage.
4. Ranking-based Black–Litterman views are constructed when signal dispersion is sufficient; otherwise, no views are applied.
5. The posterior expected returns are computed using the Black–Litterman updating formula.
6. Portfolio weights are obtained by solving a constrained mean–variance problem.
7. An exponential smoothing step is applied to limit turnover and avoid abrupt changes in allocations.

When no valid views are available, the Black–Litterman posterior collapses to the Markowitz prior, ensuring continuity and robustness of the strategy.

Overall, this implementation allows us to evaluate whether weak but systematic machine learning signals, when embedded into a disciplined Black–Litterman framework, can generate incremental performance improvements without reintroducing the instability observed in less constrained allocation approaches.

```
tau      <- 0.05
c_omega  <- 0.15
top_k    <- 2
```

```

bot_k    <- 2
q_month  <- 0.005

port_ret_BL <- rep(NA_real_, nrow(R_xts))
W_hist_BL  <- list()

w_prev_bl <- rep(1/n, n)

for (k in seq_along(rebal_dates)) {

  t_idx <- rebal_dates[k]
  if (t_idx <= max(window_days_S, window_days_mu)) next
  t_date <- as.Date(index(R_xts)[t_idx])

  R_S <- R_xts[(t_idx - window_days_S + 1):t_idx, ]
  Sigma_ann <- cov_shrink_ann(coredata(R_S), perio = perio)

  R_mu <- R_xts[(t_idx - window_days_mu + 1):t_idx, ]
  mu_hat <- colMeans(coredata(R_mu)) * perio - rf

  mu0 <- if (use_mu0_pi) equilibrium_pi(Sigma_ann, w_mkt = w_mkt, delta = delta)
  else rep(0, n)
  mu_prior <- alpha * mu_hat + (1 - alpha) * mu0

  sig_vec <- get_signal_asof(pred_tbl, symbols, t_date)
  views <- build_views_ranking(sig_vec, symbols,
                               top_k = top_k, bot_k = bot_k,
                               q_month = q_month)

  bl <- bl_posterior_mu(mu_prior, Sigma_ann, views$P, views$Q,
                       tau = tau, c_omega = c_omega)
  mu_bl <- bl$mu

  w_raw <- markowitz_mv_weights_bounds(
    Sigma_ann, mu_ann = mu_bl,
    lambda = lambda_mv,
    w_min = w_min, w_max = w_max
  )

  w <- (1 - gamma) * w_prev_bl + gamma * w_raw

```

```

w <- w / sum(w)
w_prev_bl <- w

W_hist_BL[[as.character(t_date)]] <- w

next_idx <- if (k < length(rebal_dates)) rebal_dates[k + 1]
else nrow(R_xts)
next_idx <- min(next_idx, nrow(R_xts))
if ((t_idx + 1) > next_idx) next

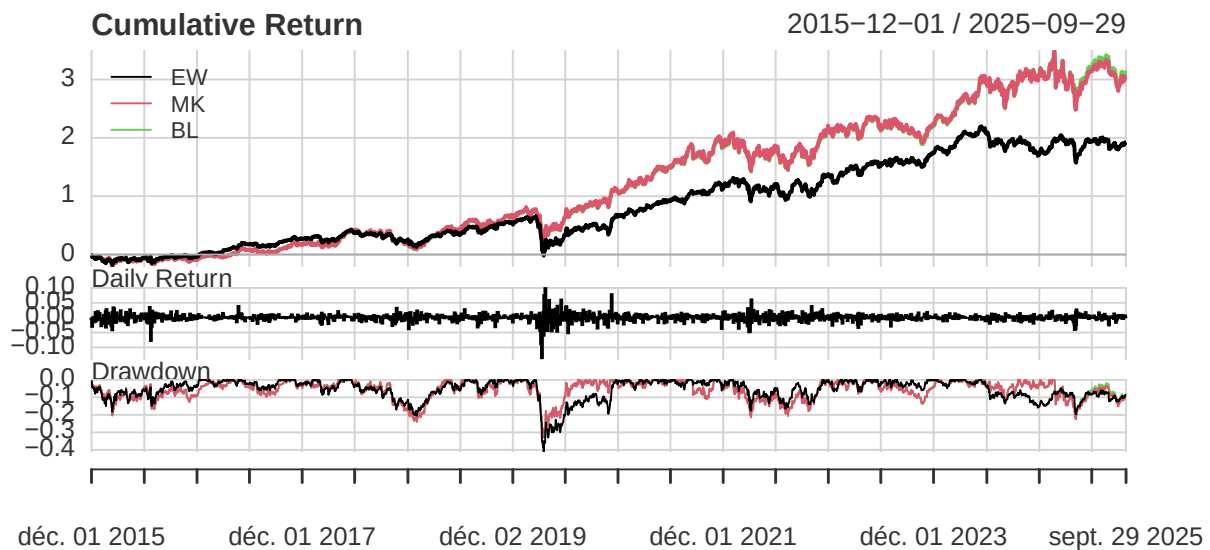
apply_range <- (t_idx + 1):next_idx
port_ret_BL[apply_range] <- as.vector(R_xts[apply_range, ] %*% w)
}

port_ret_BL_xts <- xts(port_ret_BL, order.by = index(R_xts))
port_ret_BL_xts <- port_ret_BL_xts[!is.na(port_ret_BL_xts)]
colnames(port_ret_BL_xts) <- "BL_XGB"

all3 <- merge(port_ret_EW_xts, port_ret_MK_xts, port_ret_BL_xts, join = "inner")
colnames(all3) <- c("EW", "MK", "BL")

charts.PerformanceSummary(all3,
                          main = "EW vs Markowitz vs Black-Litterman (XGB soft ranking views)"
)
```

EW vs Markowitz vs Black–Litterman (XGB soft ranking views)



```
m_EW <- perf_metrics(all3$EW, perio)
m_MK <- perf_metrics(all3$MK, perio)
m_BL <- perf_metrics(all3$BL, perio)

cat("\n=== EW ===\n"); print(m_EW)
```

```
##
## === EW ===

## $ann_return
## [1] 0.1135278
##
## $ann_vol
## [1] 0.1968622
##
## $sharpe
## [1] 0.5766866
##
## $mdd
## [1] 0.4110447
```

```
cat("\n=== MK ===\n"); print(m_MK)
```

```
##
## === MK ===
```

```
## $ann_return
## [1] 0.1501878
##
## $ann_vol
## [1] 0.2299965
##
## $sharpe
## [1] 0.6530005
##
## $mdd
## [1] 0.3296249
```

```
cat("\n=== BL ===\n"); print(m_BL)
```

```
##
## === BL ===

## $ann_return
## [1] 0.1522
##
## $ann_vol
## [1] 0.2294283
##
## $sharpe
## [1] 0.6633882
##
## $mdd
## [1] 0.329628
```

2.9.11 Performance comparison (common sample)

Over the common evaluation period, the Equal-Weight benchmark (EW) delivers an annualized return of 11.35% with an annualized volatility of 19.69%, resulting in a Sharpe ratio of 0.58 and a maximum drawdown of 41.1%.

The stabilized Markowitz strategy (MK) improves the risk–return trade-off in a meaningful way. Annualized return increases to 15.02%, with volatility rising to 23.00%, but the Sharpe ratio still improves to 0.65. Importantly, the maximum drawdown decreases substantially from 41.1% to 33.0%, suggesting that the constrained + smoothed allocation is not only more profitable but also more resilient in adverse periods.

The Black–Litterman strategy with XGBoost soft ranking views (BL) provides a small but consistent incremental gain relative to MK. Annualized return increases slightly further to 15.22%, while volatility is essentially unchanged (22.94% vs 23.00%). As a result, the Sharpe ratio improves from 0.65 to 0.66. The maximum drawdown remains virtually identical to MK (~33.0%), indicating that the ML-based BL overlay adds performance *without increasing downside risk* in this backtest.

Overall, these results are consistent with the intended design of the BL integration: the machine learning signal is treated as a low-confidence overlay, generating marginal tilts around the Markowitz

prior rather than a structural reallocation. Empirically, this translates into a modest improvement in performance metrics, while preserving the stability profile delivered by the tuned Markowitz framework.

It is important to emphasize that the Black–Litterman views are intentionally assigned low confidence in this study. This choice is consistent with the way the XGBoost model is designed and estimated. Although the model is built under strict anti-bias constraints and thoroughly backtested in a walk-forward setting, it remains deliberately conservative in terms of feature complexity and hyperparameter tuning.

As a result, the machine learning signal is expected to exhibit limited predictive power, and treating it as a strong source of information would not be economically justified. In this context, the Black–Litterman framework is used precisely as a safeguard: it allows the integration of noisy but potentially informative signals while preventing them from dominating the allocation.

That said, these results also suggest a natural extension. One may reasonably hypothesize that a more aggressively optimized XGBoost model—leveraging richer feature engineering, alternative targets, or regime-specific training—could produce more informative signals. In such a case, assigning a higher confidence level to the views within the Black–Litterman framework could lead to larger and potentially more profitable portfolio tilts.

From this perspective, the current implementation should be interpreted as a proof of concept: it demonstrates that even a cautiously tuned machine learning model, when embedded in a disciplined Black–Litterman structure, can add incremental value without degrading portfolio stability. This opens the door to further research on the interaction between model quality, view confidence, and portfolio performance.

2.9.12 Conclusion

This project revisits the classical Markowitz and Black–Litterman portfolio allocation frameworks through the lens of robustness and empirical realism. Starting from a pedagogical baseline provided during the course, the objective was not to challenge the theoretical foundations of these models, but to assess how they can be meaningfully improved and adapted to a realistic asset management setting.

A first contribution of the study lies in the stabilization of the Markowitz allocation. By extending the data horizon, moving to daily returns, introducing covariance shrinkage, imposing portfolio constraints, and adopting a rolling walk-forward rebalancing scheme, the original mean–variance framework is transformed into a dynamic and more resilient strategy. The resulting Markowitz portfolio exhibits improved risk-adjusted performance and significantly reduced drawdowns compared to a naive equal-weight benchmark, while avoiding the strong instability and regime dependence observed in unconstrained implementations.

The second contribution concerns the integration of machine learning into portfolio allocation. Rather than using predictive models as direct allocation engines, the project deliberately adopts a conservative and disciplined approach. A fully out-of-sample XGBoost model is developed under strict anti-bias constraints, including walk-forward estimation, frozen feature selection, and conservative hyperparameter tuning. As expected in such a setting, the predictive power of the model remains modest, but robust.

The Black–Litterman framework proves particularly well suited to combining these two components. By treating the stabilized Markowitz allocation as a structural prior and the machine

learning output as a low-confidence information signal, the model allows for controlled and interpretable portfolio tilts. The use of ranking-based views, small view magnitudes, and an explicit low-confidence uncertainty matrix ensures that machine learning signals enhance rather than destabilize the allocation process.

Empirically, the Black–Litterman strategy with XGBoost views delivers a small but consistent improvement over the Markowitz benchmark, without increasing volatility or drawdowns. While the performance gain is intentionally limited, it is achieved without sacrificing stability, which is precisely the objective of the proposed integration.

Overall, this work illustrates that the question is not whether machine learning can “replace” traditional portfolio theory, but how it can be embedded responsibly within well-established frameworks. The results suggest that even cautiously tuned predictive models can add value when combined with economic structure and strong risk controls. Future research could explore richer feature sets, alternative targets, or regime-dependent models, and study how increased predictive accuracy should translate into higher confidence within the Black–Litterman framework.

3 Workshop 3 — Portfolio Insurance Strategies : OBPI vs CPPI

Structured products are financial instruments created by combining multiple assets, typically a risk-free asset (such as a zero-coupon bond) with derivative instruments (options) linked to an underlying asset, such as a stock index (for example S&P 500). Their primary objective is to offer a specific return profile, often allowing participation in the performance of a risky asset while providing a certain level of capital protection at maturity. This protection is based on the bond component, while the derivatives allow the investor to gain leveraged exposure.

3.1 Strategies Overview

1.1 OBPI

The OBPI (Option-Based Portfolio Insurance) strategy aims to guarantee a minimum portfolio value by combining a risk-free asset with a call option on a risky asset. Specifically, the investor allocates a portion of their capital to a zero-coupon bond to secure the guaranteed level, and uses the remaining capital to purchase a call option on the underlying asset. This structure allows participation in market upside beyond the guaranteed level, reduced by the cost of the option premium, while providing full protection against downside risk at maturity. OBPI is a static hedging strategy: it is simple and transparent, as the protection level is set at inception and does not require dynamic adjustments during the product's life.

1.2 Floor Formula

$$F_t = Ke^{-r_f(T-t)}$$

where :

- K is the guaranteed capital at maturity
- r_f is the risk-free rate
- T is the maturity date

The value of the OBPI at time t is calculated as follows :

$$V_t^{\text{OBPI}} = F_t + \alpha, C_t \text{ with :}$$

- C_t is the value of the call option at time t
- α is the number of calls held, defined as : $\alpha = \frac{V_0 - F_0}{C_0}$

This structure ensures that the portfolio value cannot fall below the guaranteed capital at maturity.

1.3 CPPI

The CPPI (Constant Proportion Portfolio Insurance) strategy is a dynamic capital protection method based on a variable allocation between a risky asset and a risk-free asset. The central concept is the *cushion*, defined as the difference between the portfolio value and the protection floor. Exposure to the risky asset is determined by applying a multiplier to this cushion. When the market rises, the cushion increases, and exposure to the risky asset is amplified; conversely,

when the market falls, exposure is reduced to preserve the guaranteed capital. Unlike OBPI, CPPI does not rely on options but on continuous portfolio rebalancing, making it more flexible but also exposed to gap risk in case of sudden market movements.

Cushion formula :

$$C_t = V_t - F_t$$

where :

- V_t is the portfolio value at time t
- F_t is the protection floor

Exposure to the risky asset :

$E_t = mC_t$, where m is the multiplier set by the investor.

CPPI portfolio value :

$$V_{t+1} = E_t(1 + R_{t+1}) + (V_t - E_t)(1 + r_f)$$

where :

- R_{t+1} is the return of the risky asset over the period
- r_f is the risk-free rate

This rule implies that risk exposure increases when the market rises and decreases as the portfolio approaches the floor.

3.2 Modeling

2.1 Methodology, Data and Parameters

For this project, we will not use real data but instead, we will simulate a set of random variables following a normal distribution $\sim (\mu, \sigma^2)$. To implement our models, we will need some parameters that have been provided to us :

$$\hat{\mu} = 0.05$$

$$\hat{\sigma} = 0.2$$

```
mu <- 0.05 #Expected return is 5%
sigma <- 0.20 #Expected Volatility is 20%
```

To simplify the framework of our simulations, we adopt a Gaussian modeling of returns at a monthly frequency.

In this context, several parameters are introduced to structure the simulation :

```
set.seed(123)          # random seed for reproducibility
delta <- 1 / 12         # monthly time step
mat <- 1               # maturity of one year
rf <- 0.02             # risk-free rate
strike <- 100          # call strike
stress <- 0.20         # stress parameter measures risk aversion: closer to 1 means higher avers
tol <- 0
```

The parameter *delta* corresponds to the simulation time step, with returns modeled at a monthly frequency over a one-year horizon.

The variable *mat* sets the product maturity at one year.

The risk-free rate r_f is used to value the capital protection floor and the risk-free instruments in the portfolios. It is applied in the calculation of the discounted guaranteed capital as well as in the valuation of options within the OBPI strategy.

The *stress* parameter represents risk aversion in the CPPI strategy by controlling the exposure to the risky asset through the multiplier. A higher stress level leads to a more conservative allocation, while a lower stress level increases risk exposure.

2.2 Simulated Returns

To implement the modeling, we need to generate simulated returns for a hypothetical asset, taking into account the mean and volatility parameters defined above. We first create random numbers, which are then transformed into normally distributed returns.

Two types of returns are calculated :

- “returnE” returns, following the monthly mean and volatility of the asset
- Risk-neutral returns “returnPRN”, used for option valuation in the OBPI strategy

```

n <- mat / delta      # number of periods
u <- runif(n)          # uniform random numbers
S <- 100              # initial price

rf1 <- (1 + rf)^delta - 1
mum <- (1 + mu)^delta - 1
sigmam <- sigma * sqrt(delta)

returnE <- qnorm(u, mum, sigmam)
returnPRN <- qnorm(u, rf1, sigmam)

```

- n corresponds to the number of periods (12 for a monthly frequency over one year).
- u is a vector of uniform random numbers used to generate Gaussian draws via the inverse normal function.
- $rf1$, mum , and $sigmam$ represent the monthly risk-free rate, the monthly expected return, and the monthly volatility of the asset.

2.3 Simulated Price

Once the returns are simulated, we can reconstruct the evolution of the underlying asset over the entire period. The initial asset value is set to 100, serving as a common starting point for all simulations. The asset price is then updated period by period using the generated returns, allowing us to observe its trajectory over time.

Two price paths are constructed. The first corresponds to the underlying asset under the real-world measure, used to analyze the economic performance of the asset. The second is calculated under the risk-neutral measure, necessary for derivative valuation in the OBPI strategy.

```

# Moments on simulation path
rf1 <- (1 + rf) ^ delta - 1
mu1 <- (1 + mu) ^ delta - 1
sigma1 <- sigma * sqrt(delta)

# Stock simulation
r <- qnorm(u, mu1, sigma1)
rPRN <- qnorm(u, rf1, sigma1)

equity <- numeric(n + 1)
equityRNP <- numeric(n + 1)
equity[1] <- strike
equityRNP[1] <- strike

for (i in 1:n)
{
  equity[i + 1] <- equity[i] * (1 + r[i])
  equityRNP[i + 1] <- equityRNP[i] * (1 + rPRN[i])
}

```

```

}

st <- numeric(n + 1)      # vector of underlying prices
st[1] <- 100              # initial price

stPRN <- numeric(n + 1)   # vector for risk-neutral prices
stPRN[1] <- 100           # initial price

for (i in 1:n) {
  st[i + 1] <- st[i] * (1 + returnE[i])
  stPRN[i + 1] <- stPRN[i] * (1 + returnPRN[i])
}

```

2.4 Floor Simulation

The floor represents the discounted guaranteed capital at each date. It is the minimum level the portfolio must maintain to ensure capital protection at maturity.

With these variables and parameters, we can now simulate our structured products. The following sections show the evolution of OBPI and CPPI according to the different seeds assigned.

```

floor <- numeric(n + 1)   # vector containing floor values

for (i in 1:(n + 1)) {
  floor[i] <- strike / (1 + rf) ^ (mat - (i - 1) * delta) # discounted guaranteed capital
}

```

2.5 Call Simulation

With the risk-neutral trajectories established, the Call option price is calculated using the Black-Scholes model, allowing us to track the value of the derivative component in the OBPI strategy.

```

call <- numeric(n + 1)
call[n + 1] <- max(0, equityRNP[n + 1] - strike)

for (i in 1:n)
{
  d0 <-
    ((rf - sigma ^ 2 / 2) * (mat - (i - 1) * delta) + log(equityRNP[i] / strike)) /
    (sigma * sqrt(mat - (i - 1) * delta))
  d1 <- d0 + sigma * sqrt(mat - (i - 1) * delta)
  call[i] <-
    equityRNP[i] * pnorm(d1) - strike * exp(-rf * (mat - (i - 1) * delta)) *
    pnorm(d0)
}

```

3.3 Simulation of OBPI and CPPI

Once parameters, returns, and necessary variables are defined, we can simulate the two strategies.

3.1 Simulation of OBPI

```
obpi_call <- numeric(n + 1)
obpi_call[1] <- strike
gearing <- (obpi_call[1] - floor[1]) / call[1]

for (i in 1:n)
{
  obpi_call[i + 1] <- floor[i + 1] + gearing * call[i + 1]
}
```

3.2 Simulation of CPPI

```
cpqi <- numeric(n + 1)
cushion <- numeric(n + 1)
multiplier <- numeric(n + 1)
expo_exante <- numeric(n + 1)
expo_expost <- numeric(n + 1)
cpqi[1] <- strike
cushion[1] <- cpqi[1] - floor[1]
multiplier[1] <- 1 / stress
expo_exante[1] <- multiplier[1] * cushion[1]
expo_expost[1] <- expo_exante[1]
boundary_inf <- multiplier[1] * (1 - tol)
boundary_sup <- multiplier[1] * (1 + tol)

for (i in 1:n)
{
  expo_exante[i + 1] <- expo_expost[i] * equity[i + 1] / equity[i]
  cpqi[i + 1] <-
    expo_exante[i + 1] + (cpqi[i] - expo_expost[i]) * floor[i + 1] / floor[i]
  cushion[i + 1] <- cpqi[i + 1] - floor[i + 1]
  multiplier[i + 1] <- expo_exante[i + 1] / cushion[i + 1]
  expo_expost[i + 1] <-
    ifelse(
      multiplier[i + 1] < boundary_inf ||
      multiplier[i + 1] > boundary_sup,
      multiplier[1],
      multiplier[i + 1]
    ) * ifelse(cushion[i + 1] < 0, 0, cushion[i + 1])
}
```

3.3.1 Results of simulation

Before talking about simulation results, we need to define what is a seed.

Seed Definition A seed is a value used to initialize a random number generator. It makes random draws reproducible: with the same seed, the same numbers and results are obtained. Using

a seed allows consistent scenario comparison and verification of results, even with random processes.

Simulations were run for OBPI and CPPI using two different seeds (123 and 1234) to observe different evolutions of the underlying and portfolio values.

3.3.2 Seed 1234

Below, we show the evolution of the underlying asset and the two strategies using seed 1234. We can see that the values of both strategies never fall below the floor at maturity, even with a decline in the underlying between months 7 and 9. The asset price remains stable around its initial value, with two volatility spikes in months 7 and 9. These strategies allow participation in the market while simultaneously protecting capital.

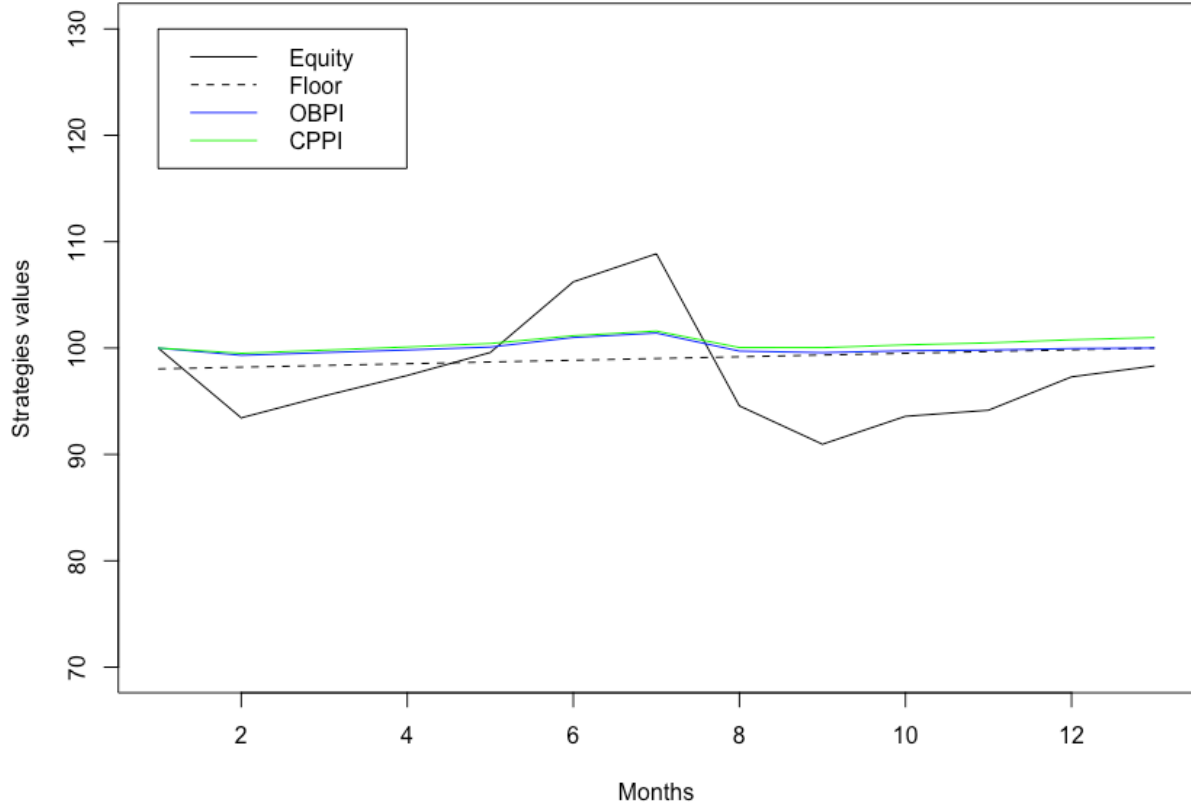


Figure 1: Trajectories evolution with seed 1234

3.3.3 Seed 123

We can now compare the value of the underlying asset as well as the two strategies using seed 123. We observe that the values of both strategies never fall below the floor at maturity. In

contrast to seed 1234, the underlying asset performs much better than in the previous simulation. Both strategies slightly benefit from the market optimism around the equity, but they do not fully capture the uptrend potential.

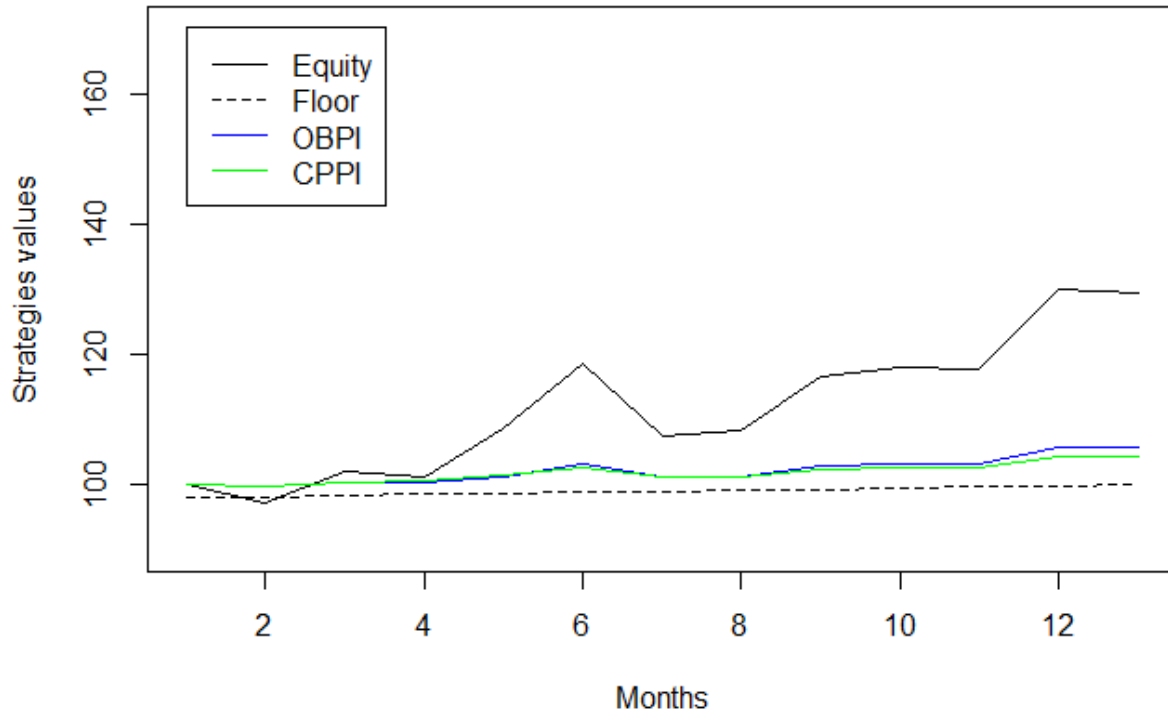


Figure 2: Trajectories evolution with seed 123

3.3.4 Conclusion :

OBPI and CPPI allow cautious investment with capital protection through the floor. They limit exposure to extreme market movements, reducing volatility and preventing very high returns. These products suit investors seeking a balance between security and moderate growth

3.4 Monte Carlo Simulation

Monte Carlo simulation is used here to analyze the behavior of capital-protection strategies, such as OBPI and CPPI, in an uncertain environment. The performance of these products strongly depends on the future evolution of the underlying asset, which is inherently random and cannot be described by a single scenario. The Monte Carlo method therefore involves generating a large number of possible asset price paths, consistent with the parameters defined at the beginning of our presentation, in order to obtain a probabilistic view of the outcomes.

In this context, 10,000 simulations are performed to approximate the final values of the equity, OBPI, and CPPI. For OBPI, the simulation allows evaluation of participation in market upside while maintaining capital protection via a guaranteed floor. For CPPI, it is essential to capture the effect of the dynamic allocation mechanism between the risky asset and the risk-free asset, depending on the cushion and multiplier. Monte Carlo thus enables comparison of these strategies not only in terms of average return, but also in terms of volatility and probability of capital protection, providing a comprehensive analysis of the risk-return trade-off.

3.1 Code Simulation Monte Carlo

```
nsimul <- 10000
n <- mat / delta
u <- matrix(0, n, nsimul)

for (j in 1:nsimul)
{
  u[, j] <- runif(n)
}

# Moments on simulation path
rf1 <- (1 + rf) ^ delta - 1
mu1 <- (1 + mu) ^ delta - 1
sigma1 <- sigma * sqrt(delta)

# Stock simulation
r <- qnorm(u, mu1, sigma1)
rRNP <- qnorm(u, rf1, sigma1)

equity <- matrix(0, n + 1, nsimul)
equityRNP <- matrix(0, n + 1, nsimul)
equity[1,] <- strike
equityRNP[1,] <- strike

for (i in 1:n)
{
  for (j in 1:nsimul)
  {
    equity[i + 1, j] <- equity[i, j] * (1 + r[i, j])
    equityRNP[i + 1, j] <- equityRNP[i, j] * (1 + rRNP[i, j])
  }
}

# Call simulation
call <- matrix(0, n + 1, nsimul)

for (j in 1:nsimul)
{
  call[n + 1, j] <- max(0, equityRNP[n + 1, j] - strike)
```

```

}

for (i in 1:n)
{
  for (j in 1:nsimul)
  {
    d0 <-
      ((rf - sigma ^ 2 / 2) * (mat - (i - 1) * delta) + log(equityRNP[i, j] /
                                                              strike)) / (sigma * sqrt(mat - (i - 1) * delta))

    d1 <- d0 + sigma * sqrt(mat - (i - 1) * delta)
    call[i, j] <-
      equityRNP[i, j] * pnorm(d1) - strike * exp(-rf * (mat - (i - 1) * delta)) *
      pnorm(d0)
  }
}

# Floor simulation
floor <- numeric(n + 1)

for (i in 1:(n + 1))
{
  floor[i] <- strike / (1 + rf) ^ (mat - (i - 1) * delta)
}

# OBPI simulation
obpi_call <- matrix(0, n + 1, nsimul)
robpi_call <- matrix(0, n, nsimul)
obpi_call[1,] <- strike
gearing <- (obpi_call[1, 1] - floor[1]) / call[1, 1]

for (i in 1:n)
{
  for (j in 1:nsimul)
  {
    obpi_call[i + 1, j] <- floor[i + 1] + gearing * call[i + 1, j]
    robpi_call[i, j] <- obpi_call[i + 1, j] / obpi_call[i, j] - 1
  }
}

# CPPI simulation
cpqi <- matrix(0, n + 1, nsimul)
rcppi <- matrix(0, n, nsimul)
cushion <- matrix(0, n + 1, nsimul)
multiplier <- matrix(0, n + 1, nsimul)
expo_exante <- matrix(0, n + 1, nsimul)
expo_expost <- matrix(0, n + 1, nsimul)
cpqi[1,] <- strike

```

```

cushion[1,] <- cppl[1,] - floor[1]
multiplier[1,] <- 1 / stress
expo_exante[1,] <- multiplier[1,] * cushion[1,]
expo_expost[1,] <- expo_exante[1,]
boundary_inf <- multiplier[1] * (1 - tol)
boundary_sup <- multiplier[1] * (1 + tol)

for (i in 1:n)
{
  for (j in 1:nsimul)
  {
    expo_exante[i + 1, j] <-
      expo_expost[i, j] * equity[i + 1, j] / equity[i, j]
    cppl[i + 1, j] <-
      expo_exante[i + 1, j] + (cppl[i, j] - expo_expost[i, j]) * floor[i +
                                                                    1] / floor[i]

    cushion[i + 1, j] <- cppl[i + 1, j] - floor[i + 1]
    multiplier[i + 1, j] <-
      expo_exante[i + 1, j] / cushion[i + 1, j]
    expo_expost[i + 1, j] <-
      ifelse(
        multiplier[i + 1, j] < boundary_inf ||
        multiplier[i + 1, j] > boundary_sup,
        multiplier[1, j],
        multiplier[i + 1, j]
      ) * ifelse(cushion[i + 1, j] < 0, 0, cushion[i + 1, j])
    rcpl[i, j] <- cppl[i + 1, j] / cppl[i, j] - 1
  }
}

# Monte-Carlo results
fin_return <- matrix(0, nsimul, 3)
price <- matrix(0, nsimul, 3)
vol <- matrix(0, nsimul, 3)
insur <- matrix(0, nsimul, 3)

for (j in 1:nsimul)
{
  price[j, 1] <- equity[n + 1, j]
  price[j, 2] <- obpi_call[n + 1, j]
  price[j, 3] <- cppl[n + 1, j]
  fin_return[j, 1] <- (equity[n + 1, j] / strike) ^ (1 / mat) - 1
  fin_return[j, 2] <- (obpi_call[n + 1, j] / strike) ^ (1 / mat) - 1
  fin_return[j, 3] <- (cppl[n + 1, j] / strike) ^ (1 / mat) - 1
  vol[j, 1] <- sd(r[, j]) / sqrt(delta)
  vol[j, 2] <- sd(robpi_call[, j]) / sqrt(delta)
  vol[j, 3] <- sd(rcpl[, j]) / sqrt(delta)
}

```

```

insur[j, 1] <- ifelse(price[j, 1] >= strike, 1, 0)
insur[j, 2] <- ifelse(price[j, 2] >= strike, 1, 0)
insur[j, 3] <- ifelse(price[j, 3] >= strike, 1, 0)
}

ret_MC <- colMeans(fin_return)
vol_MC <- colMeans(vol)
insur_MC <- colSums(insur) / nsimul

```

In this chart, which shows 200 simulated trajectories out of the 10,000, for clarity, we can see that the equity value fluctuates around its initial price of 100, with a few simulations deviating significantly from the initial price, reaching values below 80 for the worst cases and above 150 for the best.

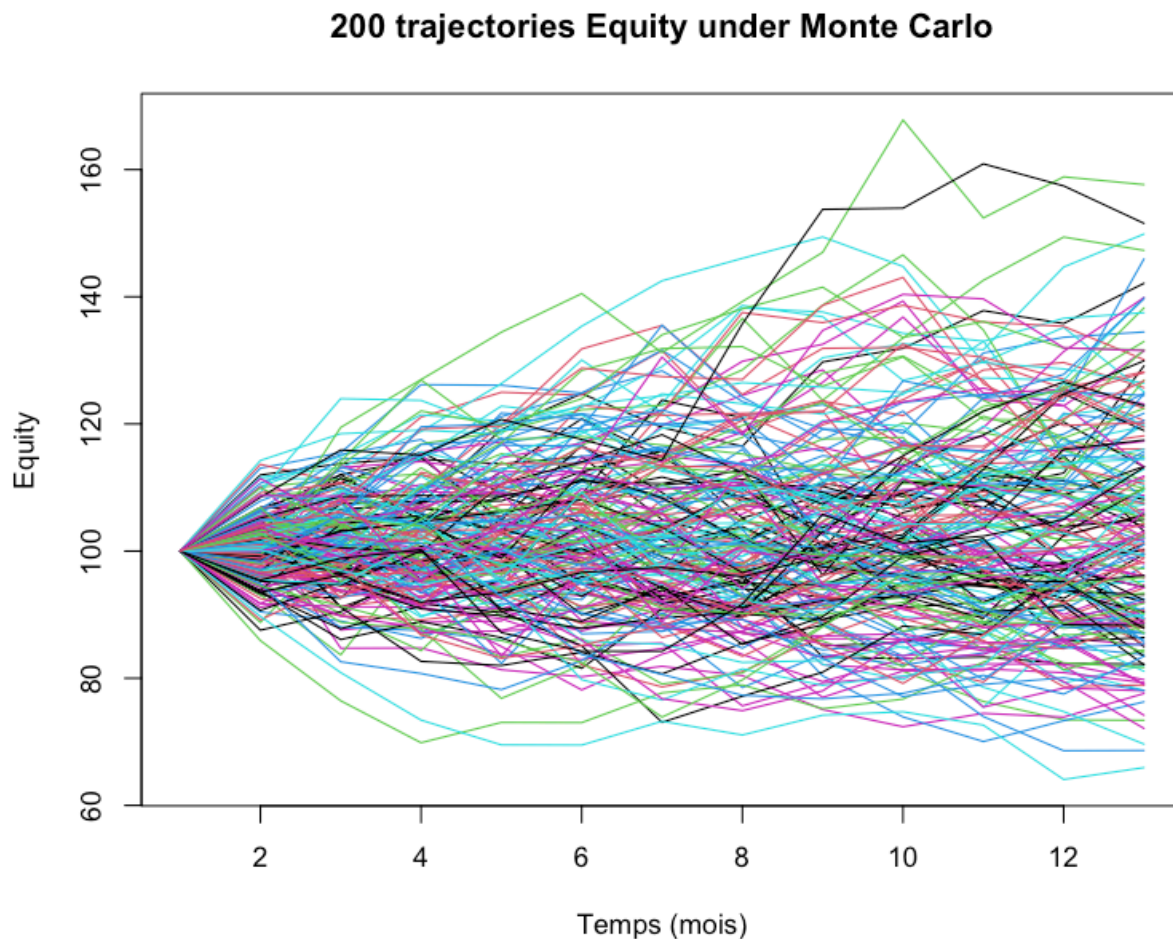


Figure 3: Monte Carlo Simulation : 200 trajectories on the 10 000 simulated

3.4.1 Monte Carlo Simulation Results

To compare the characteristics of each product, we provide an interactive table summarizing key metrics such as the annualized mean return, annualized volatility, and the insurance rate (indicating whether the product's final value is above the initial value of the underlying, 100, at the end of the period).

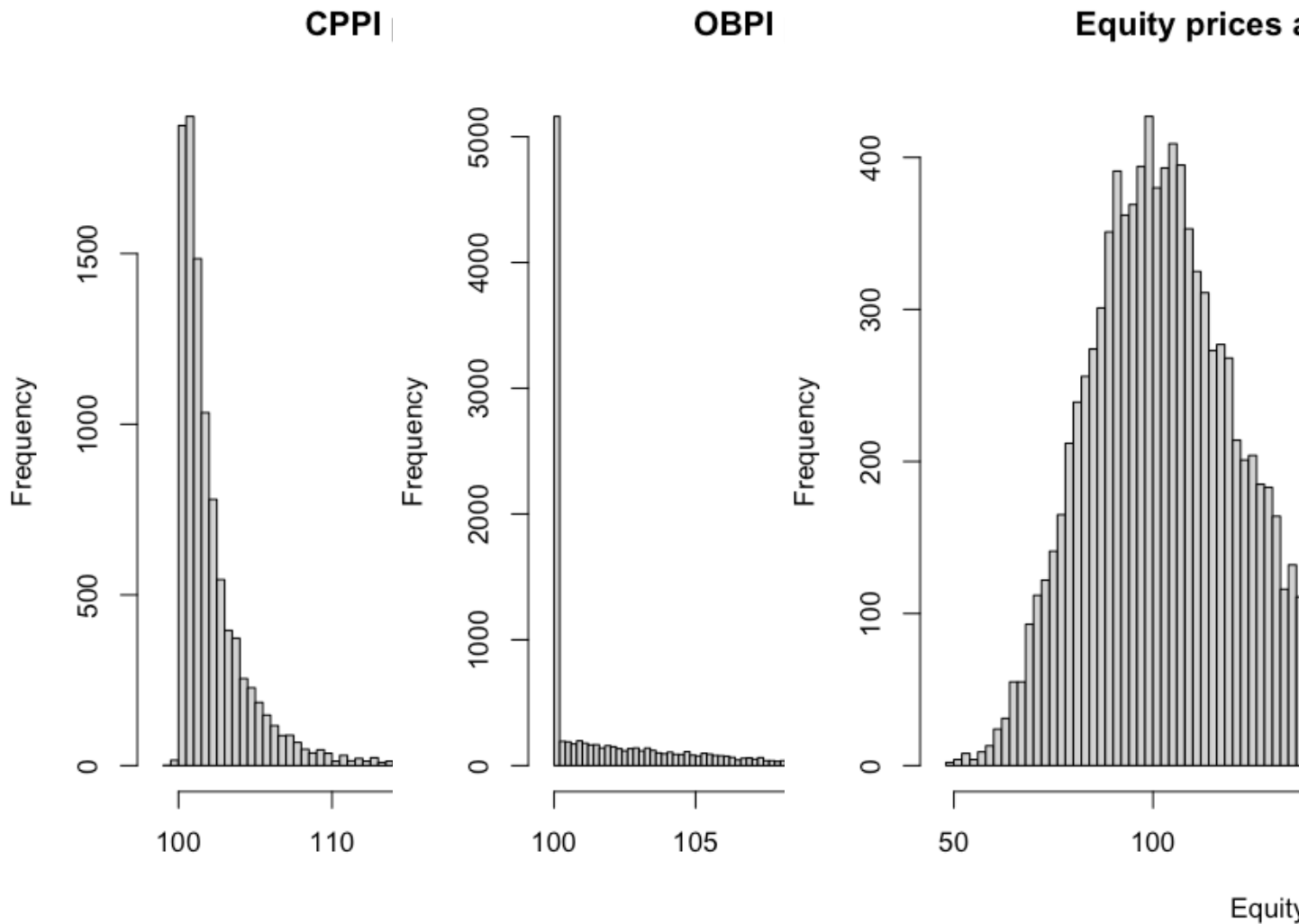
Monte Carlo results

	Equity	OBPI	CPPI
Annualised return	0.0499	0.0200	0.0233
Volatility	0.1953	0.0263	0.0218
Insurance rate	0.5586	1.0000	0.9983

Although the average return of these strategies is lower than that of the underlying, they ensure that the initial capital is preserved in almost 100% of scenarios, making them attractive solutions for a cautious investor wishing to participate in market upside without risking significant losses.

Between the two strategies, CPPI stands out for its dynamic flexibility: the multiplier and the cushion can be adjusted according to the investor's risk profile. In our simulation, with the chosen parameters, CPPI provides a slightly higher average return than OBPI while maintaining strong capital protection

The charts below help illustrate the results discussed above, showing the insurance rates of each product after our Monte Carlo simulations.



3.5 General Conclusion

In summary, OBPI and CPPI strategies allow investment in a risky asset while limiting the risk of losing the initial capital. OBPI prioritizes simplicity and security, whereas CPPI combines protection with higher performance potential through its dynamic adjustment. These products are particularly suited for investors seeking a balance between safety and participation in market growth, and the Monte Carlo analysis demonstrates the effectiveness of these mechanisms in an uncertain environment.

4 References

Clauss, P. (2025). *Workshop 3 — Portfolio insurance strategies* (course notes).

- Clauss, P. (2025). *Workshop 1 — Equity portfolio choice, Global Minimum Variance portfolio and factor models* (course notes).
- Ledoit, O., and Wolf, M. (2004). “Honey, I Shrunk the Sample Covariance Matrix.” *Journal of Portfolio Management*.
- DeMiguel, V., Garlappi, L., and Uppal, R. (2009). “Optimal versus Naive Diversification: How Inefficient is the 1/N Portfolio Strategy?” *Review of Financial Studies*.
- Jagannathan, R., and Ma, T. (2003). “Risk Reduction in Large Portfolios: Why Imposing the Wrong Constraints Helps.” *Journal of Finance*.